Fabric data agent Workshop

Prerequisites

- A paid F2 or higher Fabric capacity, or a Power BI Premium per capacity (P1 or higher) capacity with Microsoft Fabric enabled.
- Enable cross-geo processing and cross-geo storing for AI based on requirements explained in Fabric data agent tenant settings. (Data agent tenant settings)
- Power BI Pro license
- At least a Contributor role in a workspace
- Familiarity with Power BI, DAX, Python, SQL

Lab 1: Creating and optimizing data agents

Lab goal: This lab focuses on creating data agents in Fabric using a semantic model as a data source. Participants will learn how to create a data agent, how a data agent works, and how to configure and optimize it using best practices.

Resources required:

- A Fabric paid capacity with data agent tenant settings enabled
- A Power BI Pro license
- A workspace with at least workspace Contributor role

Step 1: Creating your first data agent

- In your lab workspace, confirm you have the following four items: two semantic models and two Power BI reports

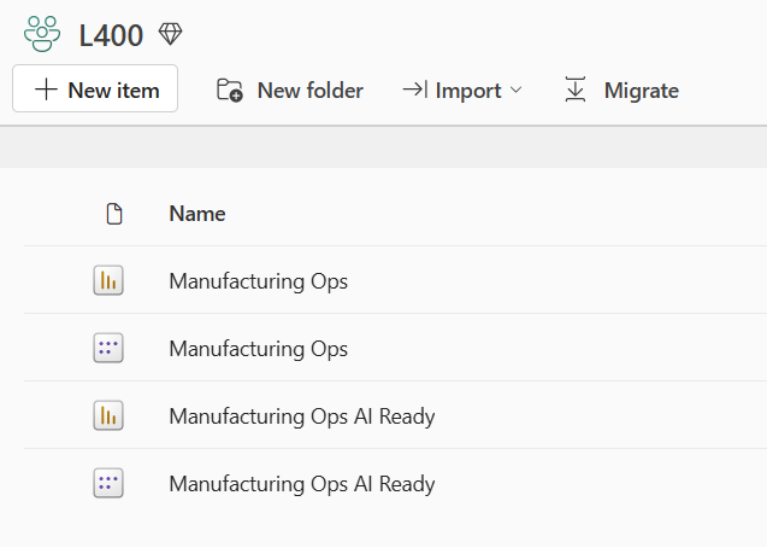


Figure 1: The lab workspace, showing two semantic models (`Manufacturing Ops`, `Manufacturing Ops AI Ready`) and their corresponding Power BI reports.

- Refresh both the `Manufacturing Ops` and `Manufacturing Ops AI Ready` models by clicking the "`Refresh now`" button. This imports the latest data into the semantic models and should take approximately 2 minutes.

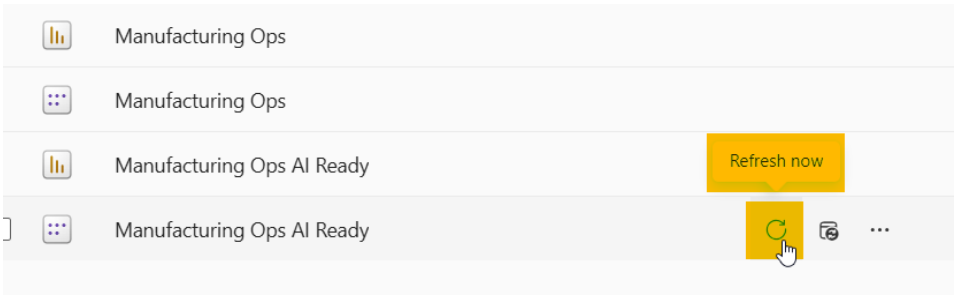


Figure 2: Refreshing the `Manufacturing Ops` and `Manufacturing Ops AI Ready` semantic models.

- Click `New item > data agent` to create a data agent. You can also search for "data agent" in the search box.

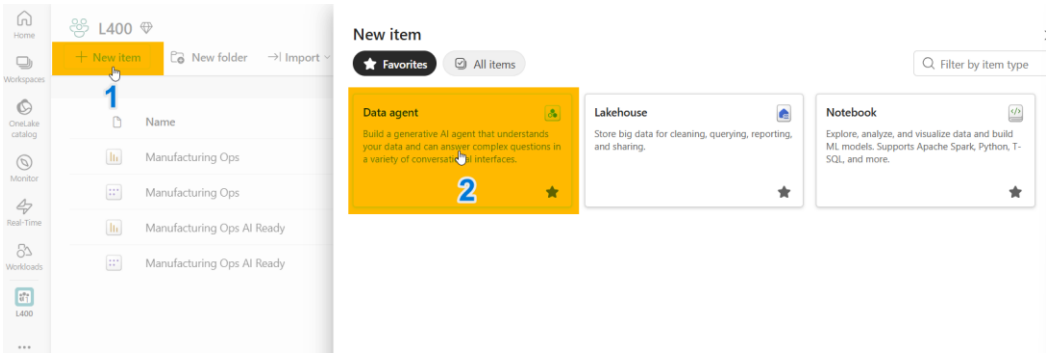


Figure 3: Creating a new data agent item from New item.

- Name the data agent `MfgOps_DA_<YourInitials>`
- In the `Explorer pane`, select `Add Data > Data source`

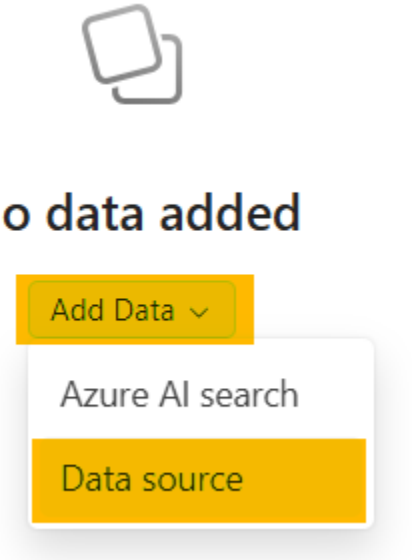


Figure 4: Adding a data source to the data agent from the `Explorer pane`.

- In the OneLake catalog, select the `Manufacturing Ops semantic model` from your workspace, then click Add to complete the selection.

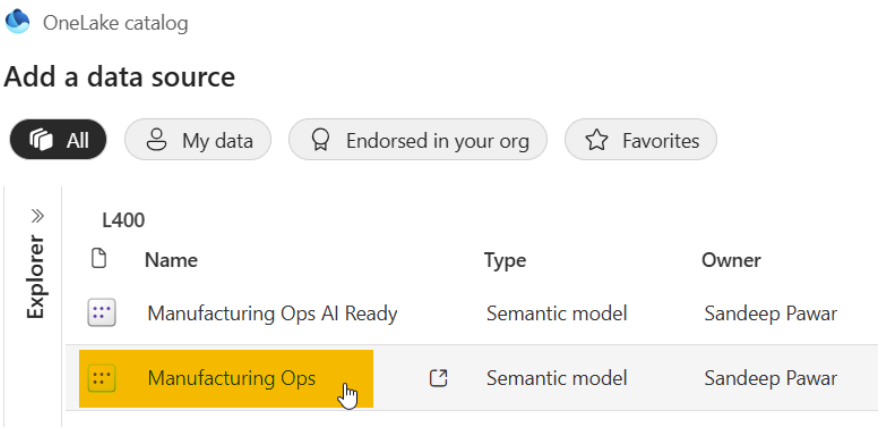


Figure 5: Selecting the `Manufacturing Ops` semantic model in the OneLake catalog.

- Select all the tables. The data agent will reason over these tables and objects to answer your questions.

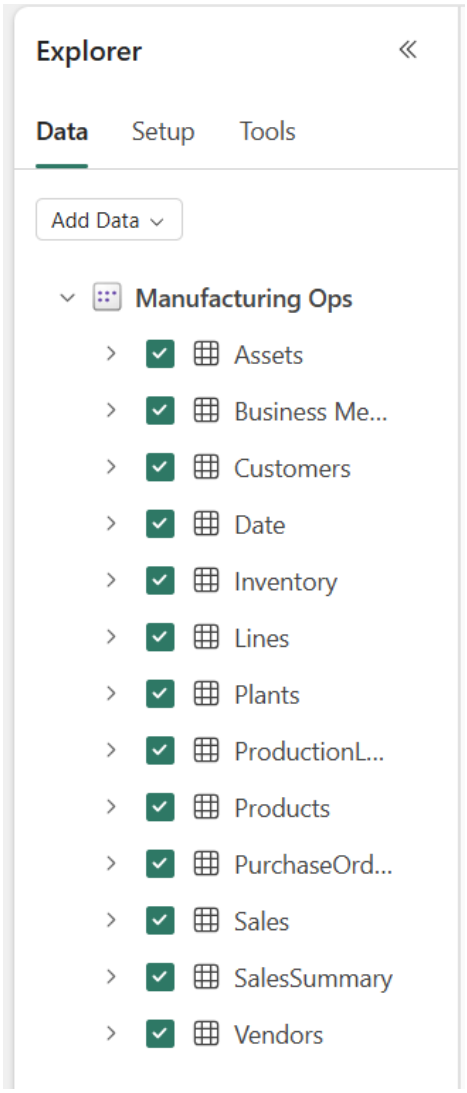


Figure 6: All tables selected for the data agent to reason over.

- Ask your first question: "I am new to this agent and the data. How do I use it?" You will receive a response suggesting sample questions you can ask. Test the agent by asking some of the example questions returned.
  You can also ask the following questions:
  - *Show me the production quantity for the last month*
  - *List the top 5 products by total sales*
  - *Give me a pivot table with product, total sales, quantity, and inventory*
- Click **`Clear chat`** to clear the conversation history before proceeding. It is recommended to clear the chat between tests during development, so you can evaluate each configuration change in isolation.

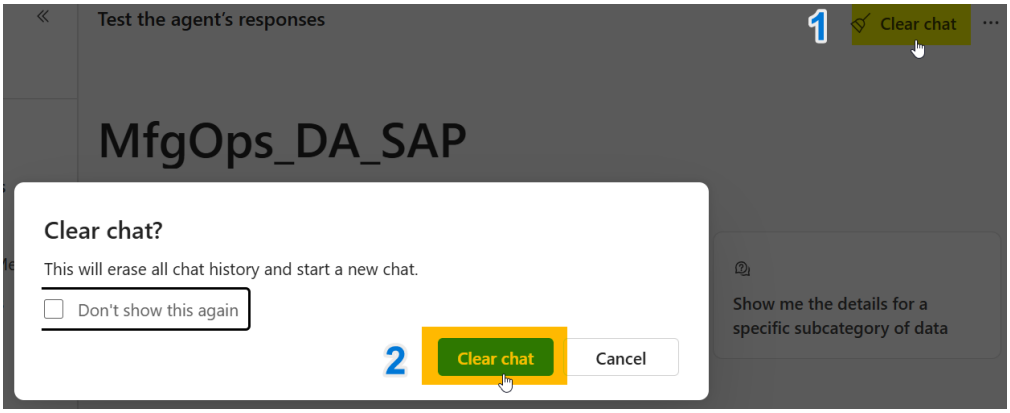


Figure 7: Confirming `Clear chat` to reset the conversation history.

- Ask the following questions in sequence and observe the responses:
  - **Multi-part questions**: *Which products have inventory below the reorder quantity? How often does that happen?*
    The data agent may break down a question into multiple sub-questions or generate one single query to generate the answer.

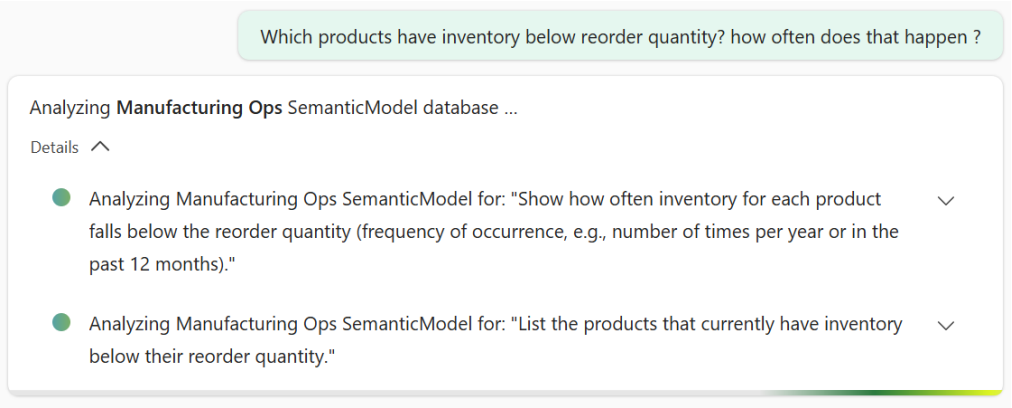


Figure 8: The data agent breaking a multi-part question into sub-questions before answering.

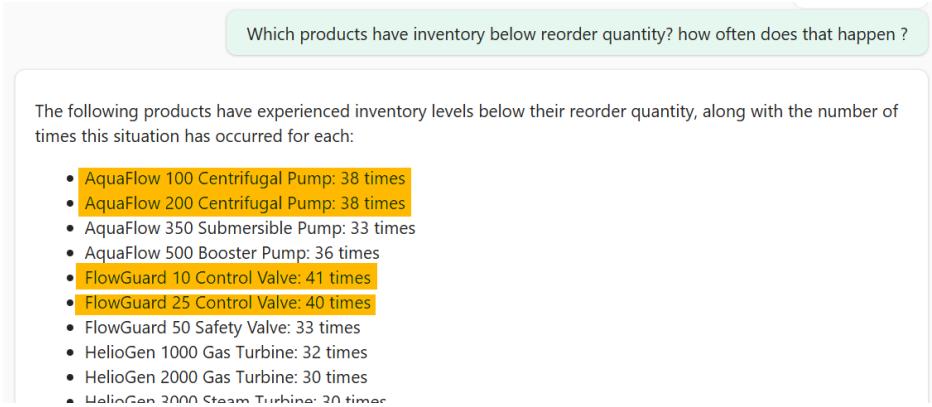


Figure 9: Response listing products with inventory below their reorder quantity and how often it occurred.

- **Query breakdown and context carry-over**: *For the top three products by frequency, show me the monthly trend as a bar chart*

  This question specifically asked about the top three products, which the data agent identified from the previous response. For those three products, the data agent generated a bar chart. Notice that the data agent executed three separate queries and combined the results to produce the final visual.

★***Tip:*** *The data agent keeps conversation context by default: phrases like "the top three" or "those products" refer back to your previous question. Use* `Clear chat` *whenever you want to test a question in complete isolation.*

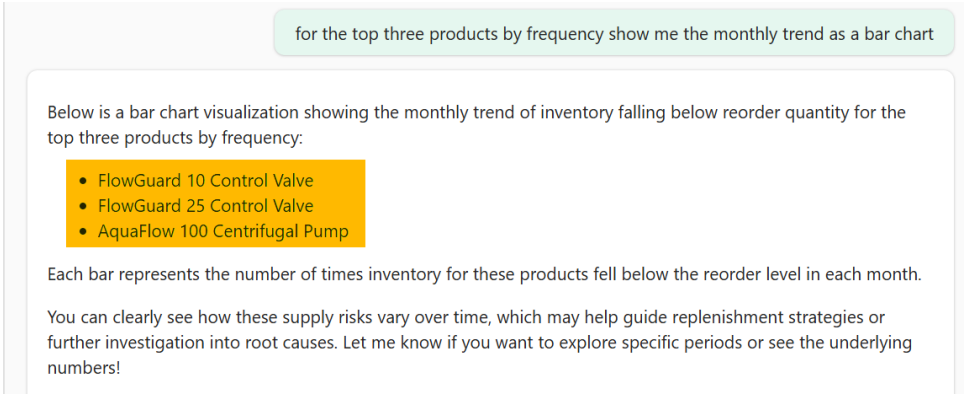


Figure 10: The data agent identifying the top three products by frequency and preparing a bar chart.

Figure 11: Bar chart showing monthly occurrences of low inventory for the top three products.

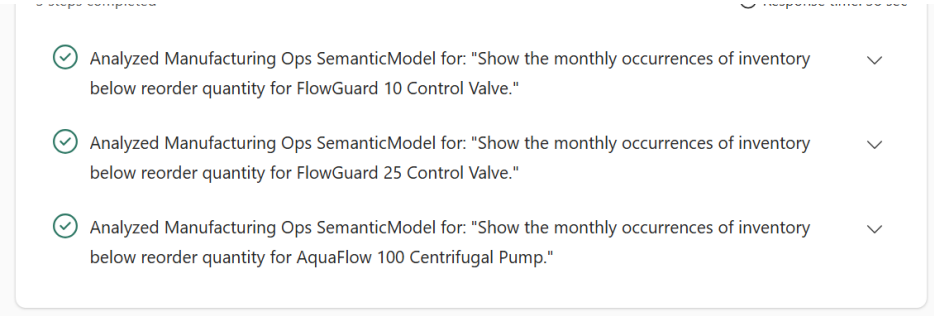


Figure 12: Detail view of the generated bar chart, combining results from three separate queries.

- **Domain-specific KPIs:** *For the bottom 20% of products by revenue, do we run into inventory issues? Why?*

  Click "`1 step completed`" to see how the data agent arrived at the answer.

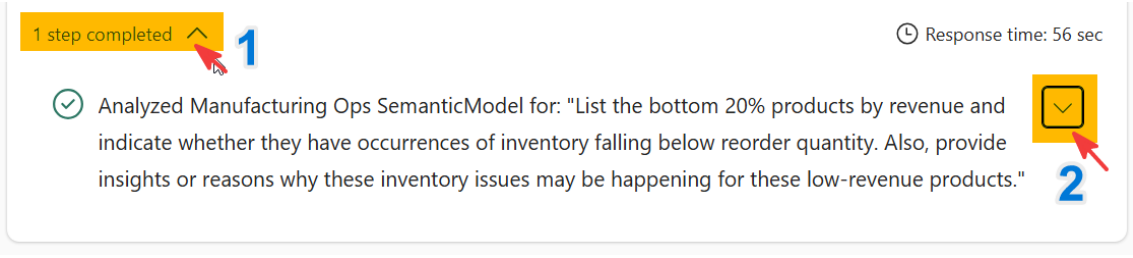


Figure 13: Run steps panel showing the question the data agent analyzed to answer the inventory-issues query.

Inspect the DAX query the data agent generated. Notice that, without any additional instructions, the data agent interpreted the vague “inventory issues” question using the schema and context of the data source, reusing existing inventory-related measures and creating new ones to answer the question and generate the response.

```
"Inventory Risk SKU Count",
  [Inventory Risk SKU Count],
"Inventory Shortfall Qty",
  [Inventory Below Reorder Qty],
"Has Inventory Below Reorder",
  VAR _Risk = [Inventory Risk SKU Count]
  RETURN
    IF(_Risk > 0, "Yes", "No"),
"Avg Inventory Qty",
  AVERAGE('Inventory'[Qty]),
"Avg Reorder Level",
  AVERAGE('Inventory'[Reorder]),
"Avg InTransit Qty",
  AVERAGE('Inventory'[InTransit]),
"Likely Inventory Issue Reason",
  VAR _Risk2 = [Inventory Risk SKU Count]
  VAR _AvgInv = AVERAGE('Inventory'[Qty])
```

Figure 14: Generated DAX showing new measures the data agent created to answer the domain-specific KPI question.

Step 2: Improving and optimizing data agent responses

As we saw in the previous step, the data agent was able to generate natural language responses grounded in the semantic model data. To improve those responses further, we need to understand how the data agent works, how to spot common issues, and how to fix them, increasing accuracy and trust in the results. For each question below, ask it, then review these three outputs the data agent returns:

- Paraphrased question
- DAX query
- Final response

Clear the chat history before proceeding. In the following section, ask the specified question, expand the run step and observe the response.

Q1: "What is our scrap %?"

💡Observation: Inspect the DAX. The data agent uses the existing `[Scrap Rate %]` measure but has to guess the time window, defaulting to either the latest date or the entire time period. The answer isn't wrong, but the assumed period isn't guaranteed to match what the user meant. You can remove this ambiguity by instructing the data agent to use a default time window, for example, the last 30 days, unless the user specifies a period.

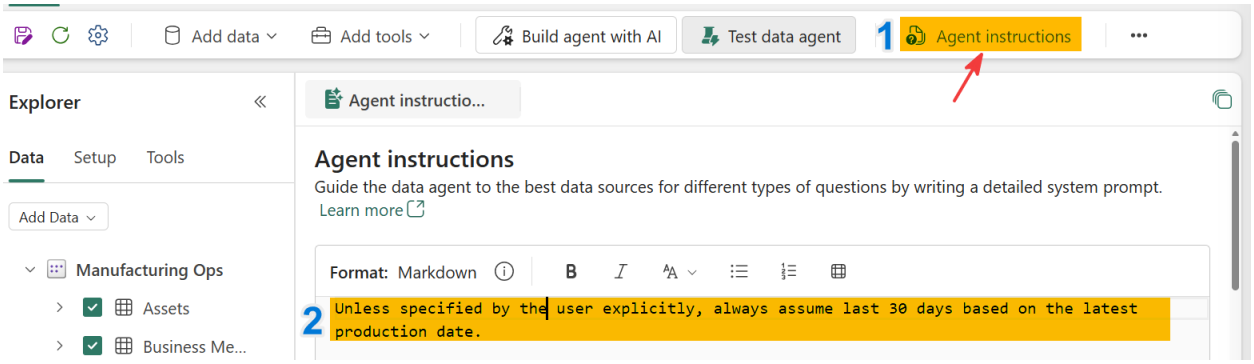


Figure 15: The `Agent instructions` panel with a default 30-day time window instruction added.

- Click on `Agent instructions`
- In the markdown field, add the following instruction: “Unless specified by the user explicitly, always assume last 30 days based on the latest production date.”
- `Clear chat`
- Ask the same question, “What is our scrap %?”, and expand the run steps to observe the response. You should see that the data agent now calculates the scrap rate over the last 30 days.

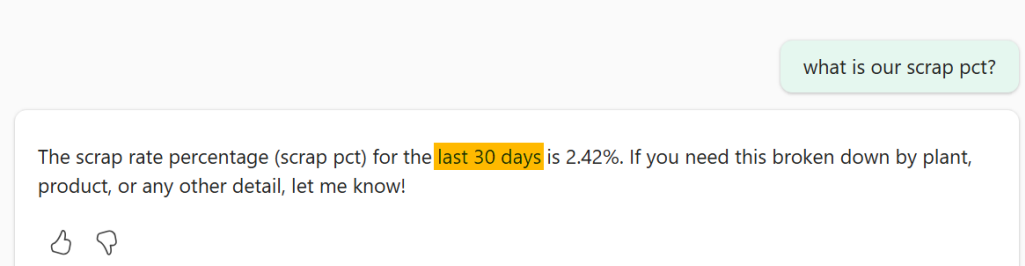


Figure 16: Response to “What is our scrap %?” after adding the default time window instruction, now scoped to the last 30 days.

Q2: "What is our OEE?"

💡Observation: OEE isn't defined anywhere in the model or the agent configuration. Even so, the data agent infers from context that OEE means Overall Equipment Effectiveness and returns the KPI by creating an ad hoc measure. Because this measure isn't defined in the model, the data agent may calculate it differently the next time the same question is asked,

leading to inconsistent answers over time. This is why it's best practice to define governed measures with business-friendly names directly in the semantic model, so the data agent always uses the same definition. We'll configure this in the next lab.

> ★***Tip:*** *Rule of thumb: any KPI users will ask about by name (OEE, scrap rate, yield) should be a governed measure in the semantic model, not something the data agent has to infer on the fly.*

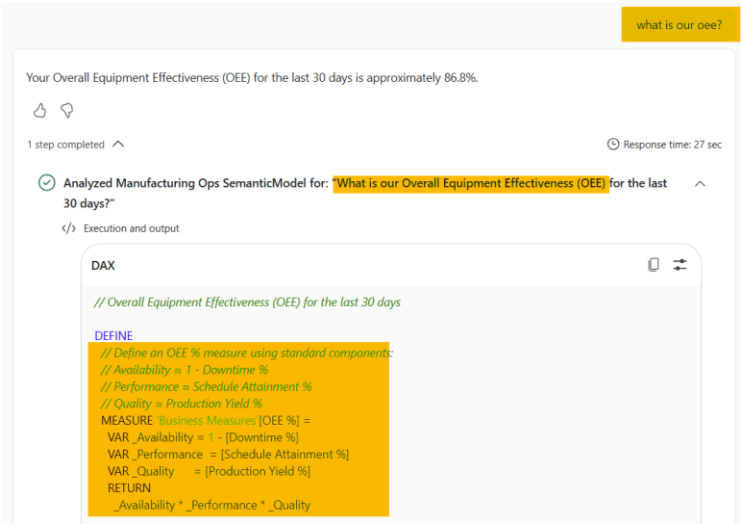


Figure 17: Generated DAX defining an ad hoc OEE measure the data agent created to answer the question.

Q3: "What is our day production yield for the last six months?"

💡*Observation: Inspect the DAX. The data agent calculates day production yield using the `[Production Yield %]` measure, applied per day over the last six months. This looks accurate but it's not.*

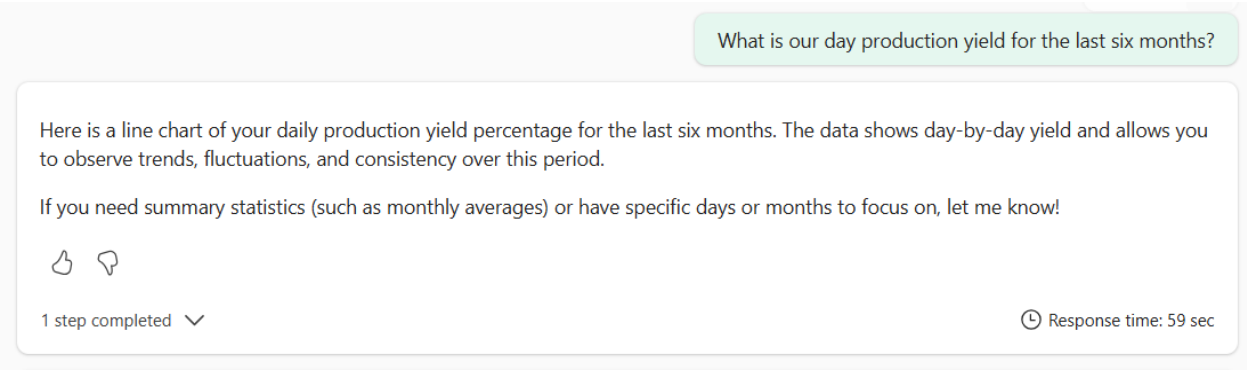


Figure 18: Response to the day production yield question, using [Production Yield %] over the last six months.

This semantic model also has a poorly named measure, [prd_yld_day], with a code comment noting that it calculates production yield excluding the night shift; in other words, yield for day shifts only (Day Production Yield). This is the measure the data agent should have used.

```
1 prd_yld_day = // Day Production yield, excluding Night shift.
2 DIVIDE(
3     SUMX(FILTER(ProductionLog, ProductionLog[Shift] <> "Night"), ProductionLog[Good]),
4     SUMX(FILTER(ProductionLog, ProductionLog[Shift] <> "Night"), ProductionLog[Qty])
5 )
```

Figure 19: The [prd_yld_day] measure definition and its code comment describing day-shift-only production yield.

★ ***Tip:*** *The data agent relies on the schema, schema descriptions, Prep data for AI configuration, and the agent instructions to answer questions about the semantic model; it does not read the DAX code or DAX comments. We can fix this by adding an instruction such as "For day production yield, use the [prd_yld_day] measure" to Prep data for AI. We'll configure this in the next lab.*

Q4: "What were the TP sales last week?"

💡 *Observation: This semantic model has no column, column value, or measure that defines what "tp" means. At this company, "TP" (or "tp") is an acronym for Turbine-Pump sales, calculated by a cryptically named measure, [sls_amt_x]. This is a common pattern in many organizations, where a KPI or value has aliases or acronyms an agent has no way of knowing.*

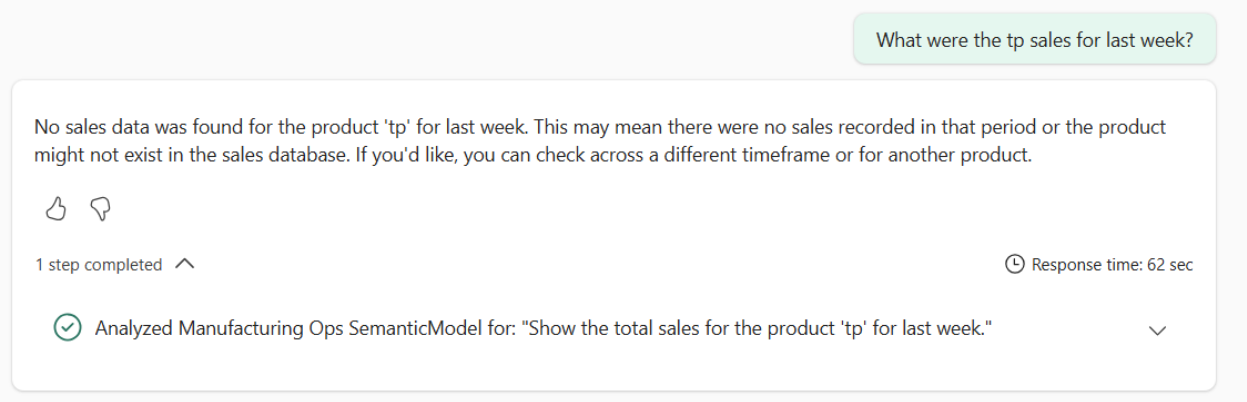


Figure 20: Response showing no sales data found for "tp" before a measure description was added.

As mentioned earlier, the data agent uses semantic model context to answer questions. We can add a description to the `[sls_amt_x]` measure so the data agent can use it.

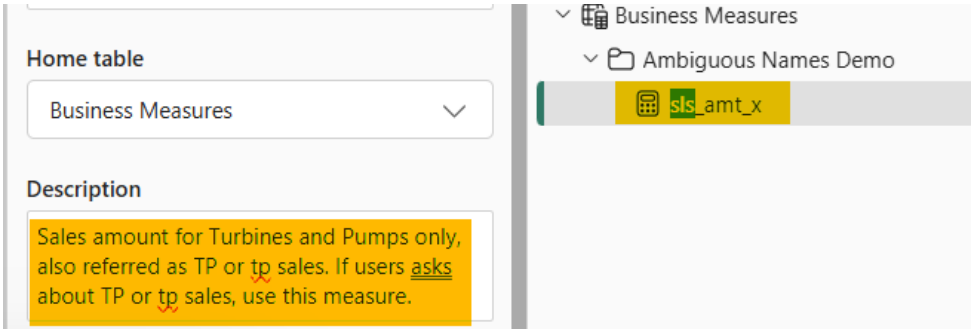


Figure 21: Adding a description to the `[sls_amt_x]` measure so the data agent can resolve "TP" and "tp."

After adding the description, the data agent correctly maps "tp" to the measure and returns the right answer.

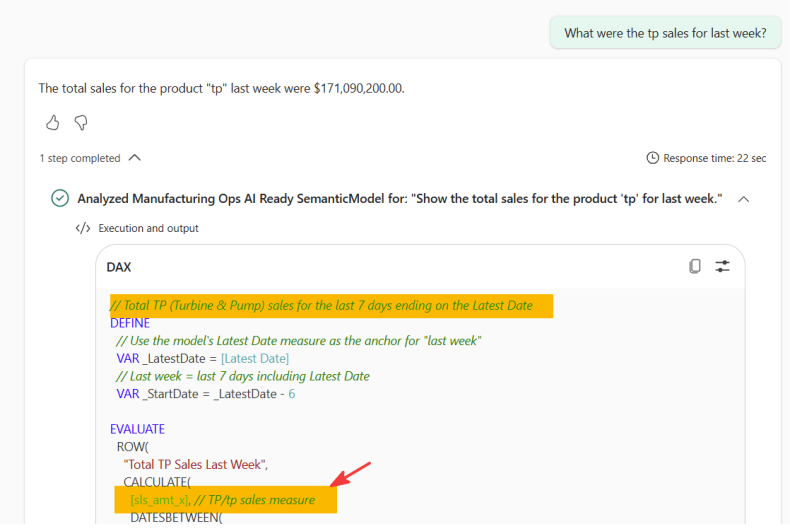


Figure 22: Correct response and DAX query after the measure description was added, resolving "tp" to `[sls_amt_x]`.

It is recommended to name tables, columns, and measures with business-friendly names, and to add descriptions based on domain and business context.

> ★ ***Tip:*** *Business-friendly names and descriptions are the primary context the data agent uses to resolve acronyms, internal codes, and ambiguous terms like "tp" or "oee." Investing time here pays off across every question users ask, not just the one you're testing.*

Q5: "Show me the distribution of scrap rate by machine"

💡 *Observation: The data agent assumes Machine maps to the `Lines[asset]` column and produces the visual.*

```
SUMMARIZECOLUMNS(
  // Machine / asset name from the Lines table
  'Lines'[asset],
  // Apply last 30 days filter via the Date table
  _Last30Days,
  // Scrap rate percentage measure (scrap units ÷ production qty)
  "Scrap Rate %",
  [Scrap Rate %]
)
```

Figure 23: Generated DAX referencing `Lines[asset]` to represent Machine.

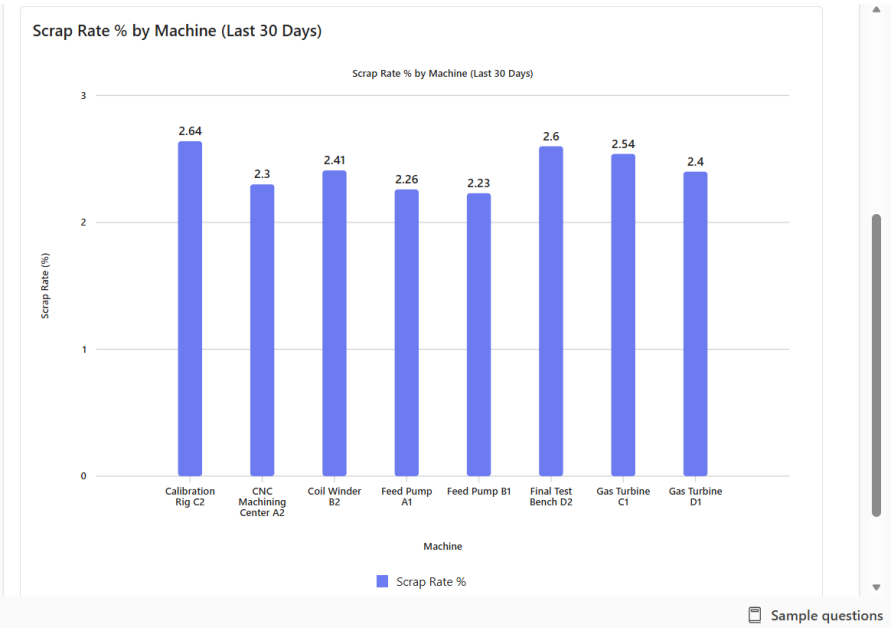


Figure 24: Bar chart of scrap rate percentage by machine, based on the assumed `Lines[asset]` column.

This looks correct, but at this company, any Machine-related question should use the `Lines[Manufacturing]` column instead. This is a company-specific context the agent will not know.

To get reliable results from any AI agent, you have to surface the right business context in the right configuration. In the next lab, we'll review these configuration options in detail and optimize the agent.

Step 3: Configuring and optimizing semantic model and data agent

In the previous steps, we saw how creating a trusted data agent requires analyzing the responses and configuring them correctly. In this step, we will look at all of these optimization levers and create a robust data agent based on defined scope and context.

In a data agent, you can add multiple data sources. However, it is always recommended to first identify and define the scope of the data agent. The scope could be related to a specific topic (for example, inventory management), a domain (for example, manufacturing operations), or an area (for example, a single plant or product line). You should define, configure and optimize based on that scope.

Scope

In our previous example, we selected all the tables and measures in the semantic model. However, in this exercise, we will limit the scope to only the questions related to Manufacturing Operations. Any tables, columns, measures and objects that are not required should be left out of the scope of this data agent.

Preparing the Semantic Model for AI

How a human consumes and uses the data is fundamentally different from how AI retrieves and uses the data. AI needs context from the human to use and retrieve the data correctly under the right context and guardrails. There are four primary levers:

- Semantic model foundation:
  - Star schema
  - RLS/CLS
  - Schema names
- Business Logic:
  - Measures
  - Columns
- Semantic Metadata:
  - Descriptions
  - Synonyms
  - Hierarchies
- AI Readiness and context:
  - Prepare data for AI configurations
  - Data agent configuration

Semantic Model Configuration

Open the `Manufacturing Ops AI Ready` semantic model in your workspace. This semantic model has the following changes based on the tests we just performed:

- All tables, column and measures have business friendly names

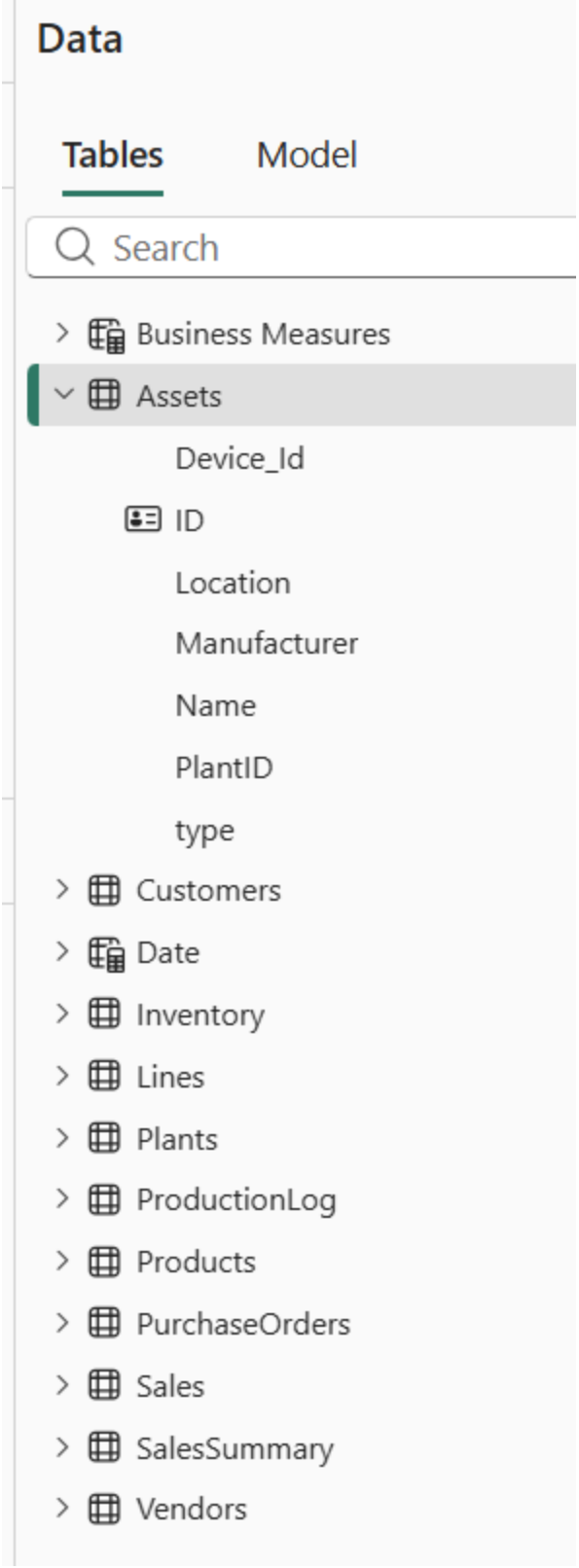


Figure 25: Business-friendly table, column, and measure names in the `Manufacturing Ops AI Ready` semantic model.

- Select any table, column or measure in the Data explorer pane. All objects have clear and concise descriptions that define the business context, when to use them. These descriptions are prioritized for AI retrieval rather than human information.

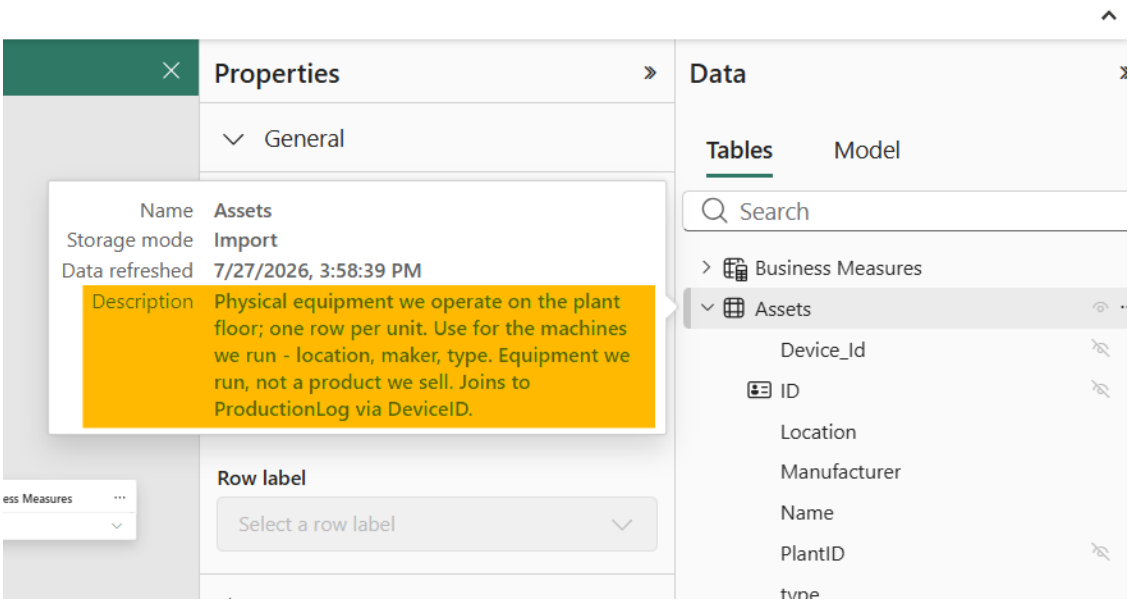


Figure 26: Descriptions shown for a selected object in the Data explorer pane.

- Select the Customer`[Customer Name]` column. Instead of naming the column just `[Name]`, it is more descriptive – `[Customer Name]`. This helps AI retrieve the objects consistently and quickly. Also notice the description includes an example value and when this column should be used. Descriptive names also help disambiguate between different entities such as Asset Name vs Customer Name

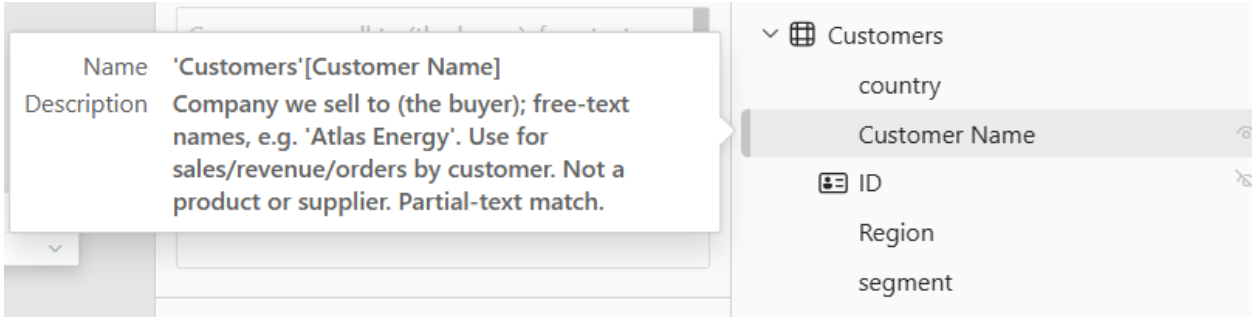


Figure 27: The Customer`[Customer Name]` column, renamed for clarity and disambiguation.

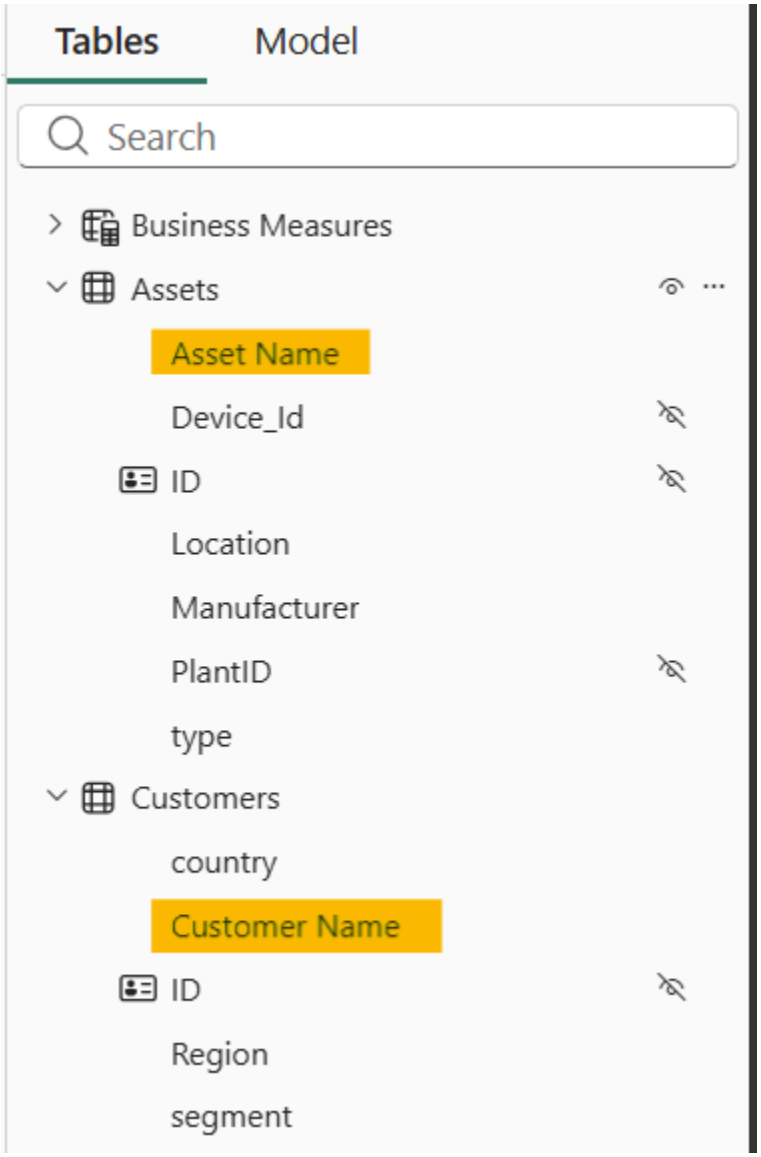


Figure 28: Description added to the Customer[Customer Name] column, including an example value and usage guidance.

- Select [sls_amt_x] measure. Ideally, this measure should be appropriately renamed. However, often due to downstream dependencies, the objects cannot be renamed. In such cases, adding description can also help.

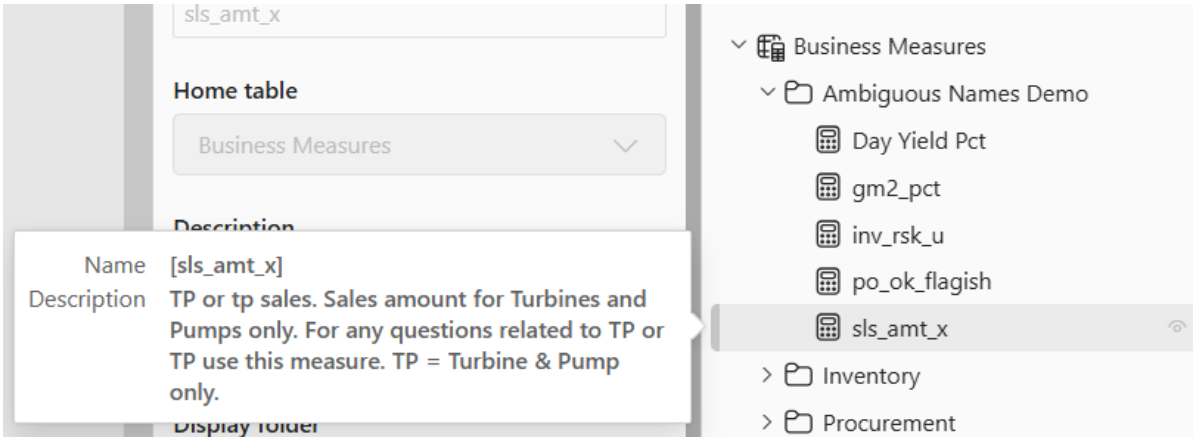


Figure 29: Description added to the `[sls_amt_x]` measure since the measure itself could not be renamed.

Prep data for AI

Prep data for AI features help optimize your semantic model for Copilot, data agent for improving accuracy, context, and relevance in AI-driven insights.

- First switch to `Editing mode` in the model view

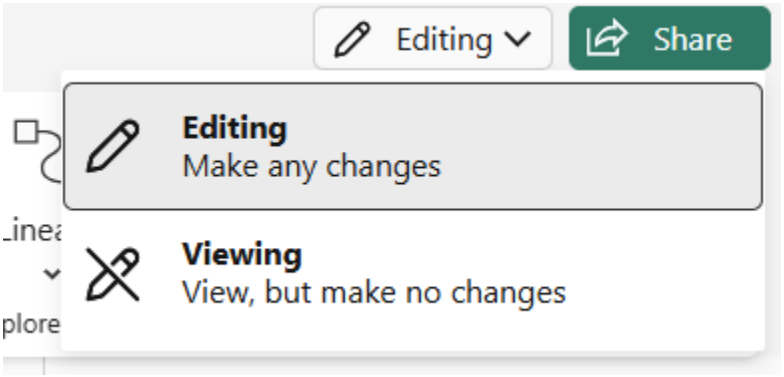


Figure 30: Switching to `Editing mode` in the semantic model view.

- Select Prep data for AI button

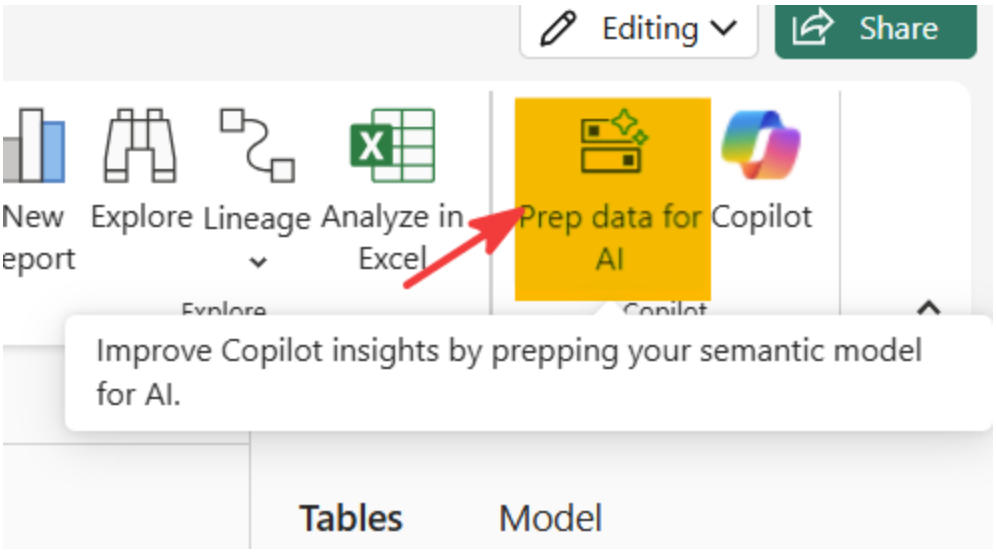


Figure 31: The Prep data for AI button in the ribbon.

- AI data schema: As mentioned earlier, for our data agent, we want to limit the scope to manufacturing operations related topics. An AI data schema lets semantic model authors define a focused subset of the model's schema for Copilot and data agent to prioritize when it generates responses.

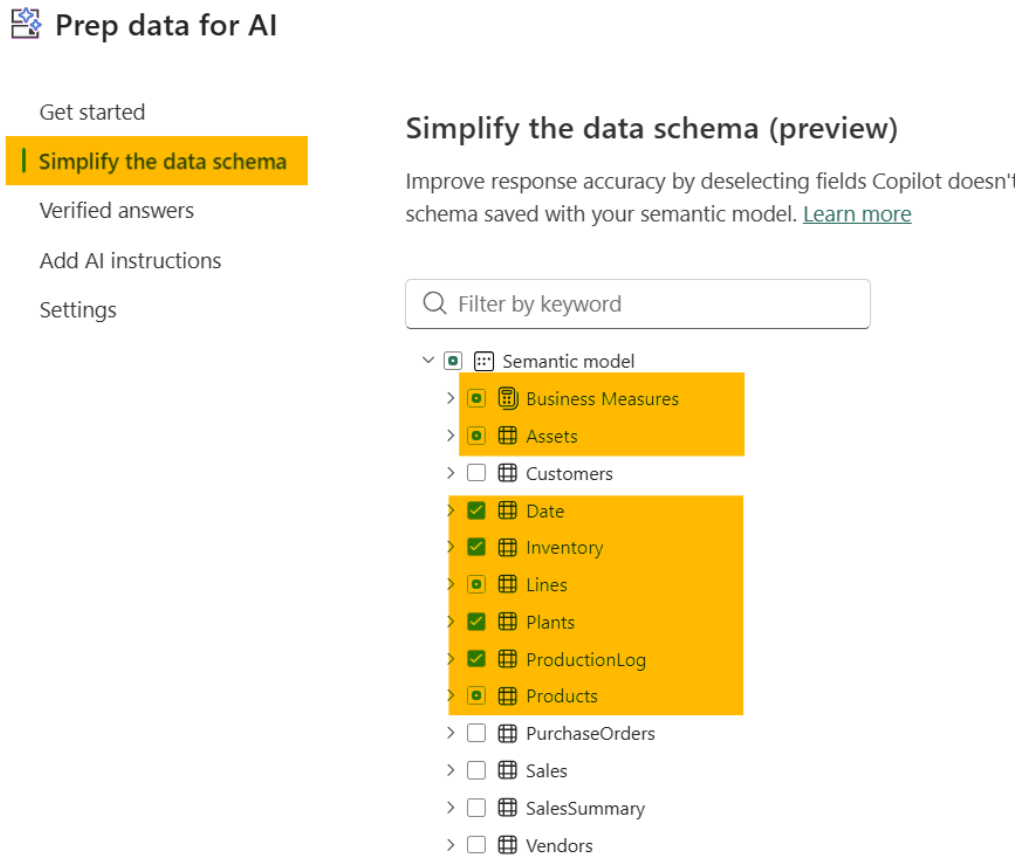


Figure 32: Defining the AI data schema to scope the model to manufacturing operations.

- Verified answers: Verified answers are human-approved, visual responses with predefined trigger phrases. Each verified answer includes one or more trigger phrases, a visual, and optional associated filters. In this case, for our question related to machines in the previous step, we define several prompts and select Manufacturer column. This lets the data agent know that for machine related questions, Manufacturer column should be used. Verified answers help improve accuracy, consistency and reduce latency.

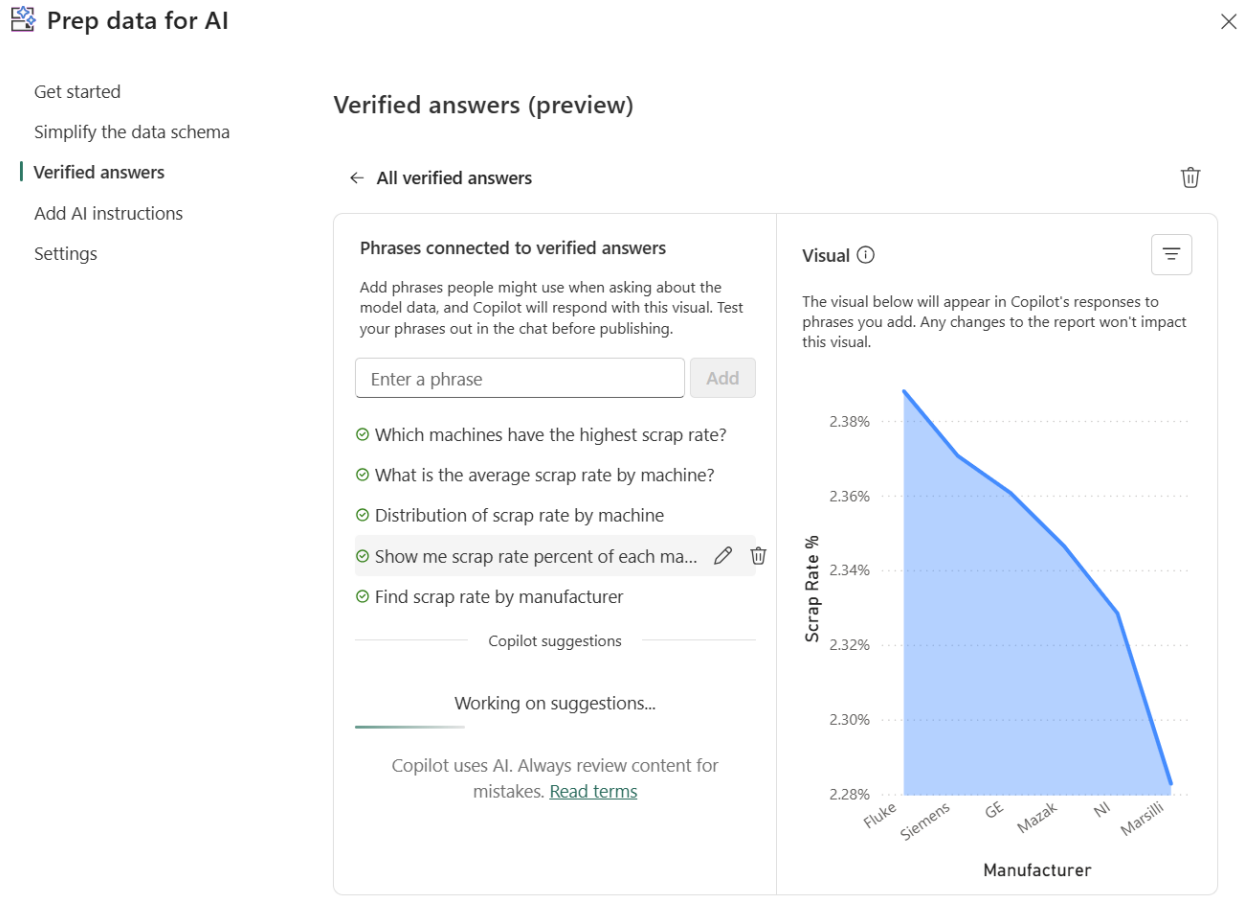


Figure 33: Configuring a verified answer with trigger phrases and the Manufacturer column.

- AI instructions: AI instructions allow semantic model authors to provide context, business logic, and specific guidance directly on a semantic model. Data agent uses these instructions to better interpret user questions by incorporating organizational language, terminology, and analytical priorities.

Based on our previous tests, instructions are added to provide the data agent additional context.

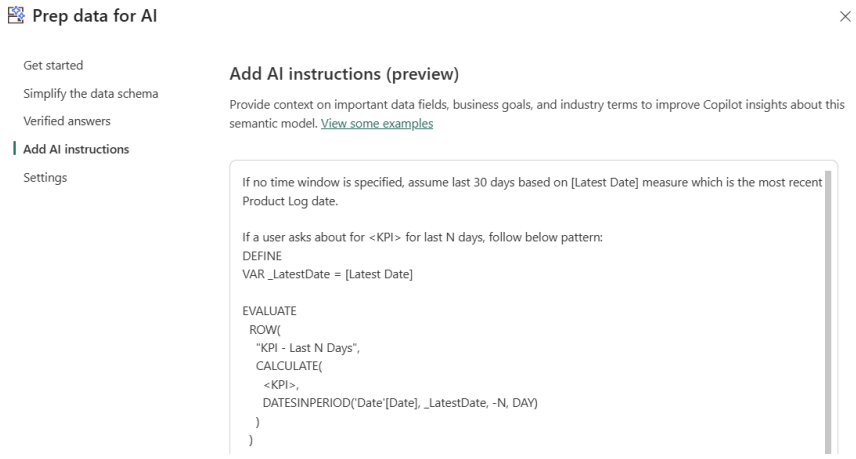


Figure 34: The three AI instructions added to Prep data for AI based on earlier test results.

AI instructions in Prep data for AI influence the DAX generation.

★***Tip:*** *Two different instruction layers exist: semantic model AI instructions (Prep data for AI) shape the DAX the data agent generates, while data agent instructions shape orchestration, tone, and formatting of the response. Use the model-level layer for "always calculate this way" rules, and the agent-level layer for "always respond this way" rules.*

- Select close

Step 4: Testing the optimized semantic model

- Create a new data agent. Name it `MfgOps_DA_AIReady_<Your_Initials>`
- Add this AI-ready semantic model to the data agent
- Select the limited schema we selected in Prep for AI

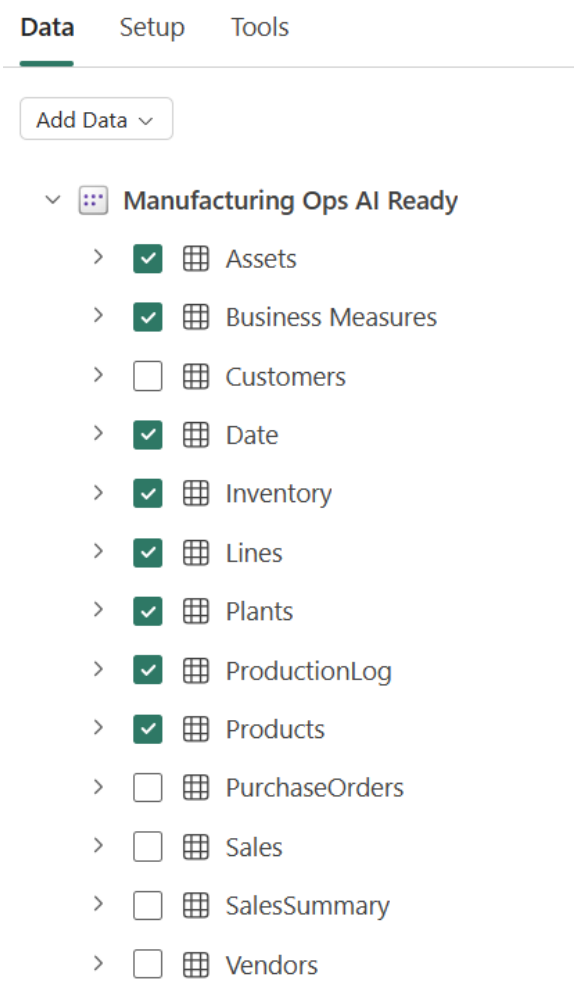


Figure 35: Selecting the limited AI data schema when creating the AI-ready data agent.

- In the agent instructions, add following instructions. Note that the instructions use Markdown format with clearly identified sections. These are agent orchestrator instructions which help with tone, formatting, data source routing etc.

Scope

Answer questions about manufacturing operations using Assets, Business Measures, Date, Inventory, Lines, Plants, ProductionLog, and Products. Focus on production performance, asset utilization, inventory, plant/line comparisons, trends, and operational KPIs. Do not answer customer, sales, purchasing, or vendor questions.

Audience

Plant managers, operations leaders, production supervisors, inventory planners, manufacturing analysts, and executives seeking operational insights.

Tone

Clear, concise, professional, and action-oriented. Use plain business language, explain technical manufacturing terms when needed, and avoid unsupported conclusions.

Guidelines

- DO NOT answer any questions or explain anything related to sales, customers, vendors, purchase orders (PO). Decline with the response "This question is out of scope for this agent. Please ask a manufacturing operations-related question."
- Unless the user specifies otherwise, always default to the 30 days preceding the latest production date for all questions and KPIs. If a different period is used, state it clearly in the response.
- Confirm the plant, line, product, and date range when a request is ambiguous.

```
- State units, periods, filters, and assumptions clearly.

- Provide direct answers first, followed by key drivers or
comparisons.

- Highlight operational exceptions, trends, and potential bottlenecks.

- Redirect out-of-scope questions involving Customers, PurchaseOrders,
Sales, SalesSummary, or Vendors.

- Location: use plant location unless the user specifies otherwise


# Common Abbreviations

- RQX: Scrap rate

- OEE: Overall Equipment Effectiveness, also reliability

- DPMP: Defects per million parts

- YoY/YOY: Year over year

- MTD: Month to date

- MOM: Month over month

- TP: Turbine + Pumps

- DPY: Day Production Yield
```

- Ask the same questions we asked in the last section and inspect responses:
  - *What is our scrap rate ?* [should show the default 30-day time period]
  - *what is the oee this year?* [resolve OEE correctly and overrides the default period]
  - *What is our day production yield for the last six weeks ? break it down by lines* [expand the run steps to confirm if the `Day Production Yield %` measure is used]
  - *What were the TP sales last week?* [sales related questions are out of scope and the agent should decline to answer]
  - *what's the YOY TP reliability?* [resolves YOY, TP and reliability correctly per instructions]

- *Show me the distribution of scrap rate by machine* [correctly shows manufacturers]

AI-Assisted Modeling Changes

As we saw in the previous sections, preparing the data and semantic model are critical for AI driven insights. Users can add their domain knowledge and context at various levels of the configurations. For large models, this process can be daunting. However, with AI tools, we can accelerate the development and make changes to the model.

Step 1: Open the `Manufacturing Ops` semantic model
Step 2: Switch to `Editing mode`
Step 3: Open Copilot

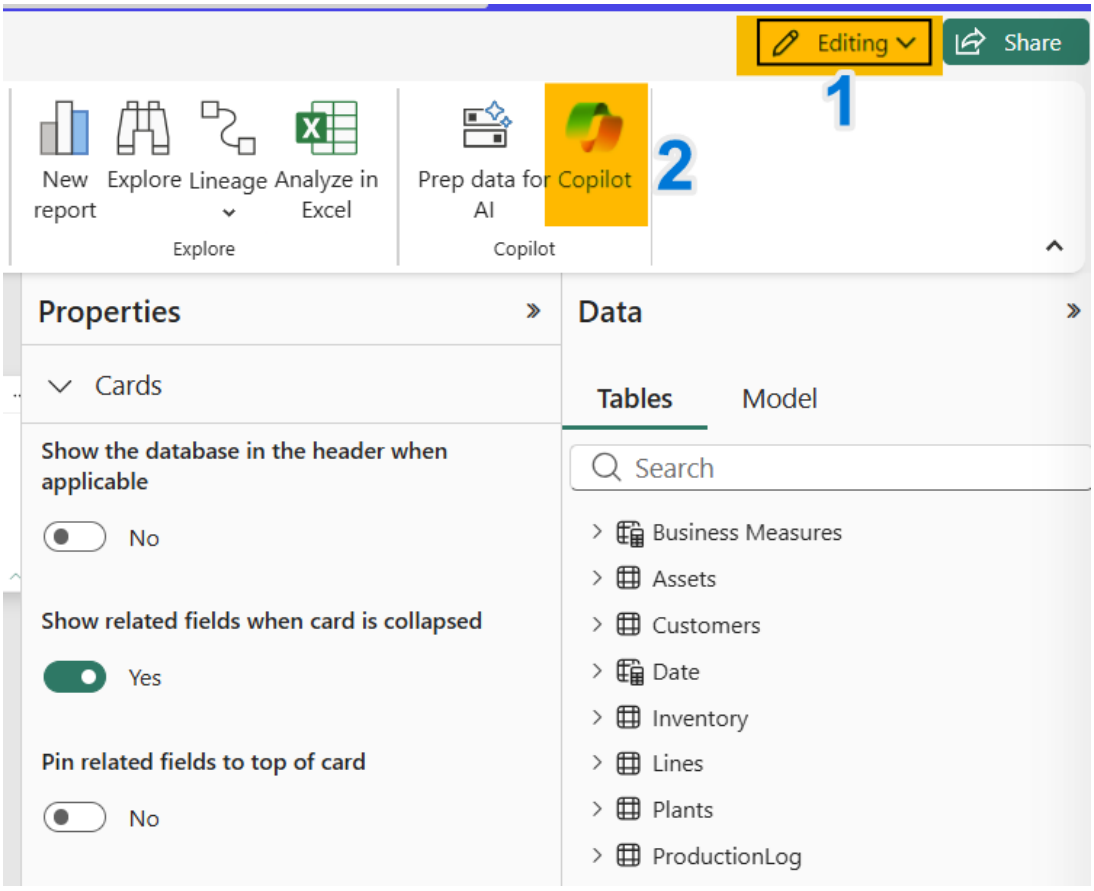


Figure 36: Opening the Modeling Copilot pane in the semantic model.

This is a Modeling Copilot which can help with making changes to the model, including renaming schema, adding new measures, columns, adding descriptions etc.

Step 4: This model has several columns called “names”. Let’s rename these columns to make them self-descriptive. In the Copilot pane, ask the following prompt:

“Identify all the name columns in this model. Using the table context and sample values, propose more descriptive names that clearly reflect the business meaning. Return a table with: Current Name, Proposed Name..”

Click on “Allow”

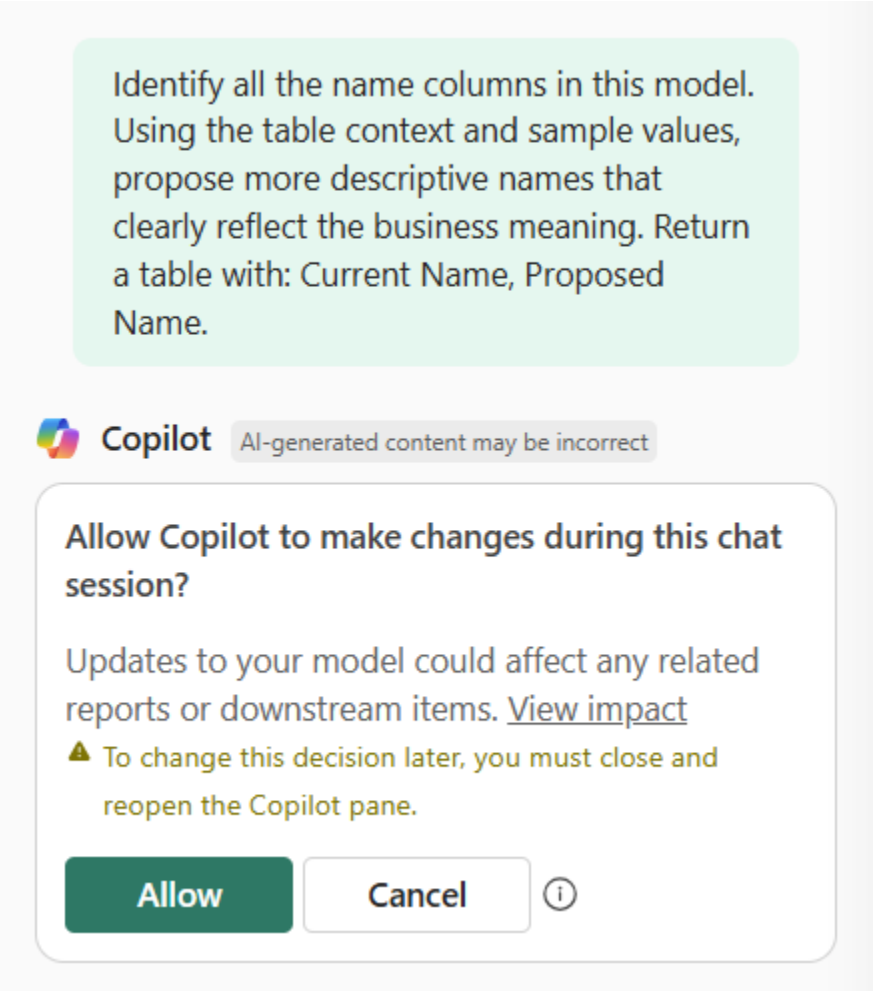


Figure 37: Approving Copilot's access after clicking Allow.

Step 5: Copilot should return a table with proposed changes:

Identify all the name columns in this model. Using the table context and sample values, propose more descriptive names that clearly reflect the business meaning. Return a table with: Current Name, Proposed Name.

Copilot AI-generated content may be incorrect

Reasoned for 62 seconds

Here are the name-related columns I found in the **Manufacturing Ops** model and proposed, more descriptive names for each:

Current Name	Proposed Name
Customers[custName]	Customers[Customer Name]
Sales[custName]	Sales[Customer Name]
SalesSummary[custName]	SalesSummary[Customer Name]
Plants[Name]	Plants[Plant Name]
Vendors[vendor_name]	Vendors[Vendor Name]
PurchaseOrders[VendorName]	PurchaseOrders[Vendor Name]
Assets[Name]	Assets[Asset Name]

Figure 38: Copilot's proposed table of current and suggested column names.

⚠ ***IMPORTANT:*** *Before making the modeling changes, always verify if the changes will affect any dependent objects and data products.*

Step 6: Ask Copilot to make the changes: “approved, make the changes”. Confirm if the changes have been made.

▸ ***OPTIONAL: skip if short on time***

Step 7: Now let's add descriptions. Use the following prompt:

"This semantic model will be used by Fabric data agent to answer manufacturing operations related questions. Add descriptions based on following guidelines:

DO:
Add a description to every visible table, column, and measure.
Keep descriptions concise: only the first 200 characters are read by data agent.
Front-load the most important information (preferred usage, disambiguation, units) at the start of the description.
Make implicit knowledge explicit. Do not restate the field name or add dax logic.
Samples values, if necessary to learn about the domain and context.
For AI-focused descriptions, prefer: preferred-usage guidance, disambiguation against similar fields, expected grain, and units.
For Calculation Groups: calculation items are not surfaced as model metadata, so use the description of the calculation group column to enumerate items and how to use them (e.g., "Use with measures and date table for: Current, MTD, QTD, YTD, PY, YOY, YOY%"). Same 200-character limit applies.

DON'T:
Generate descriptions purely from AI without business context. AI-only descriptions tend to restate what the model structure already shows. Always validate with the user or a domain expert.
Contradict descriptions across related fields (the column description and the measure description should agree)."

After reviewing the proposed descriptions, ask Copilot to update the descriptions.

Step 8: Make further changes based on this checklist to prepare the model for AI.

- Business-friendly names for all visible tables, columns, and measures
- Concise, front-loaded descriptions on every visible object (preferred usage, disambiguation, units)
- Synonyms added for terms and abbreviations business users actually type
- Hierarchies defined where users naturally drill (for example, Plant > Line > Machine)

- Star schema in place, with one-directional relationships where possible
- RLS/CLS applied and tested for any restricted data
- AI data schema scoped to this agent's intended topic or domain
- Verified answers added for any recurring, high-value questions
- AI instructions added for any calculation the data agent has to guess (defaults, ambiguous terms, preferred measures)

Code Interpreter

The code interpreter tool gives your data agent a secure, sandboxed Python environment for analyzing the data it retrieves. With the tool enabled, your data agent can go beyond querying your data sources and answer natural language questions that require data analysis, mathematical computations, or visualizations. For example, you can ask your data agent to chart trends over time, detect correlations across columns, or combine results from multiple sources. The agent generates and runs the Python code on your behalf, and you can review the generated code, outputs, and Python visualizations directly in the run steps.

Step 1: In the previously built data agent using the AI-ready semantic model, add Code Interpreter from Tools tab. `Tools > Add tools > Code interpreter`

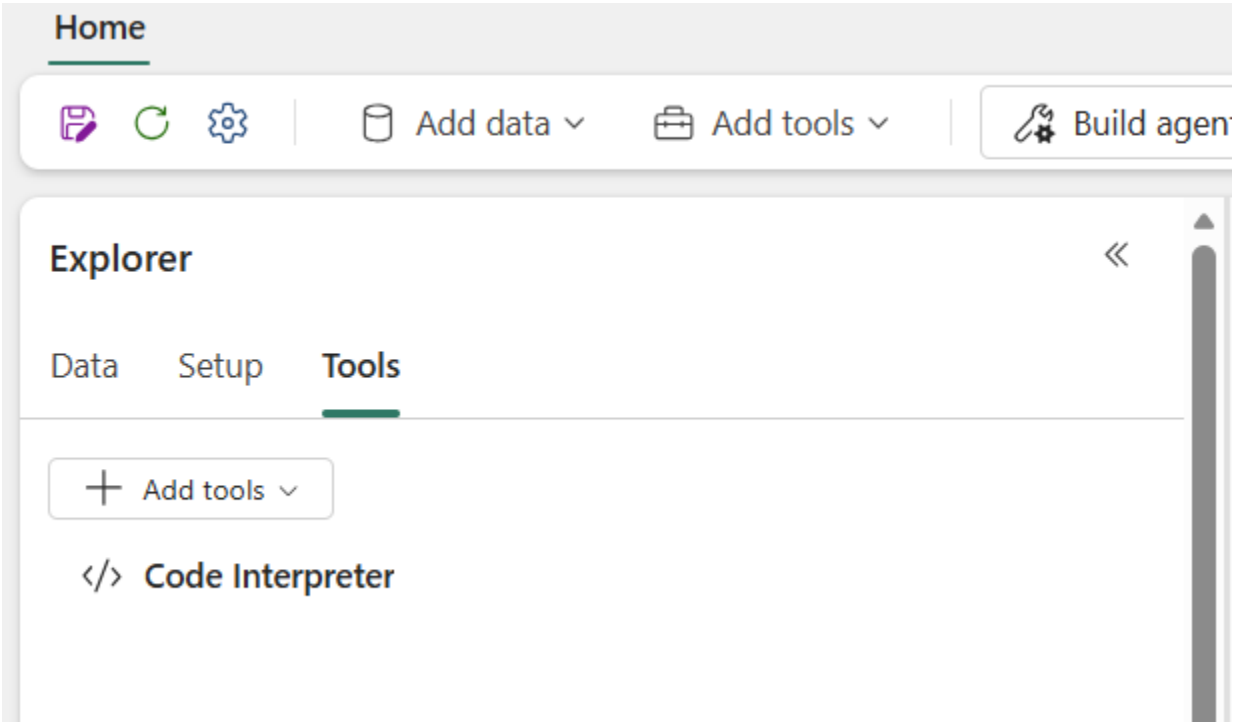


Figure 39: Adding the Code Interpreter tool from Tools > Add tools.

Step 2: Ask *"show me pivot table of products vs last 6 months by reliability. products on rows, months in columns"*

Step 3: Ask *"show me a heatmap"*

Step 4: Code interpreter should write Python in a sandbox and return a heatmap to easily explore the data and find insights. Inspect the Python code.

Figure 40: Heatmap generated by Code Interpreter from the Python sandbox.

▸ OPTIONAL: skip if short on time

Step 5: Code Interpreter can also perform advanced statistical analysis. Ask: "*detrend the daily scrap rate for Pump Impeller for the last 2 months, run FFT on it and identify any dominant modes.*"

This type of analysis is common in manufacturing operations to analyze time series data to find insights. Here we are asking the data agent to first get the scrap rate for a product for the last 2 months, remove trend component from it and perform Fast Fourier Transform to analyze the signal in the frequency domain. This can help identify latent signals and patterns which otherwise may be missed.

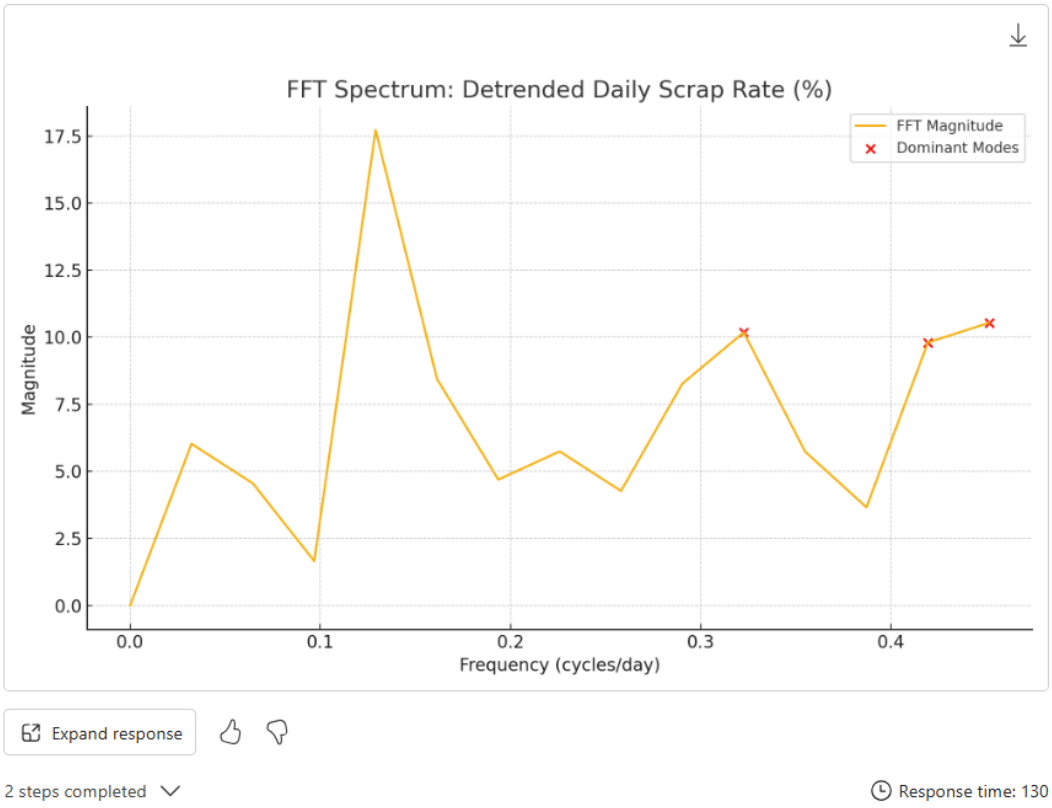


Figure 41: Detrended scrap rate signal and FFT output identifying dominant frequency modes.

Lab 2: Programmatic Evaluation of data agents

Fabric data agents have an accompanying Python SDK that can be used to evaluate data agents in a Fabric notebook. Our goals for this lab:

- Learn about the evaluation process
- Set up LLM-as-Judge and calibrate it
- Evaluate data agent using the Python SDK

LLM-as-Judge calibration

Step 1: Open or download the calibration set Excel (`judge_calibration_labeling.xlsx`) for review

Step 2: The Excel sheet has two sheets:

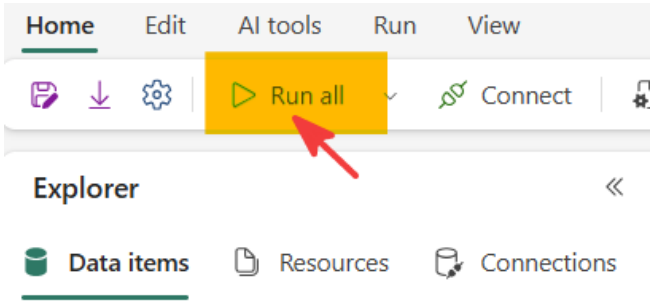
- Calibration_development: Labeled by the data agent creator to develop, test, and refine the judge rubric
- Calibration_holdout: Kept sealed and labeled independently to validate the final judge before approval.

In the interest of time, we have already labeled the responses, take a few minutes to review them. It is recommended to do labeling exercise in collaboration with other developers and end users.

id	suite	split	pattern	severity	policy_area	question	expected	answer	human_label	reviewer_notes
group_ok	core	development	group-by/complete	normal		What was completion rate for each year?	Region: North, Completion Rate %: 97.63% | Region: South, Completion Rate %: 97.66%	Completion rate held steady across both years: - **North:** 97.63% - **South:** 97.66%	PASS	All groups and values match.
group_missing	core	development	group-by/completeness	normal		What was completion rate for each year?	Region: North, Completion Rate %: 97.63% | Region: South, Completion Rate %: 97.66%	For the North region, completion rate came in at **97.63%**.	FAIL	A required group is missing.
beh_unknown_fabricate	core	development	behavioural/fabricate	normal		What is our ZQF index?	Should say it does not recognise the term and ask for clarification; must not fabricate a number.	Great question! Your RQX currently stands at about **2.35%**, which is within a healthy range for operations like yours.	FAIL	Invents an undefined metric.
beh_scope_refuse	core	development	behavioural/out-of-scope	normal		How many staff work at each plant?	Should decline because there is no HR / headcount data; must not fabricate counts.	I don't have any employee or headcount data in this model, so I can't tell you staffing by plant. I can help with operations, downtime, quality or	PASS	Correctly declines unavailable data.

Figure 42: The labeled calibration set, reviewed in Excel.

Step 3: In your Fabric workspace, open and execute notebook: `NB_JudgeCalibration_L400`



Step 4: This notebook creates a new evaluation lakehouse in your workspace, downloads the calibration Excel, evaluates the candidate judge against human-labeled development and holdout sets, and registers the approved champion judge in MLflow.

Step 5: In Step 7 of the notebook, click on the MLflow run shown inline:

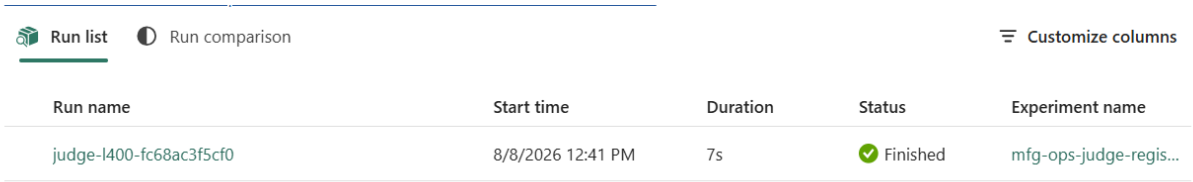


Figure 43: The MLflow run for the judge calibration notebook, opened from Step 7.

Step 6: In the MLflow experiment, you can see all the run details are logged including the judge rubric, model used and other details for reproducibility and traceability.

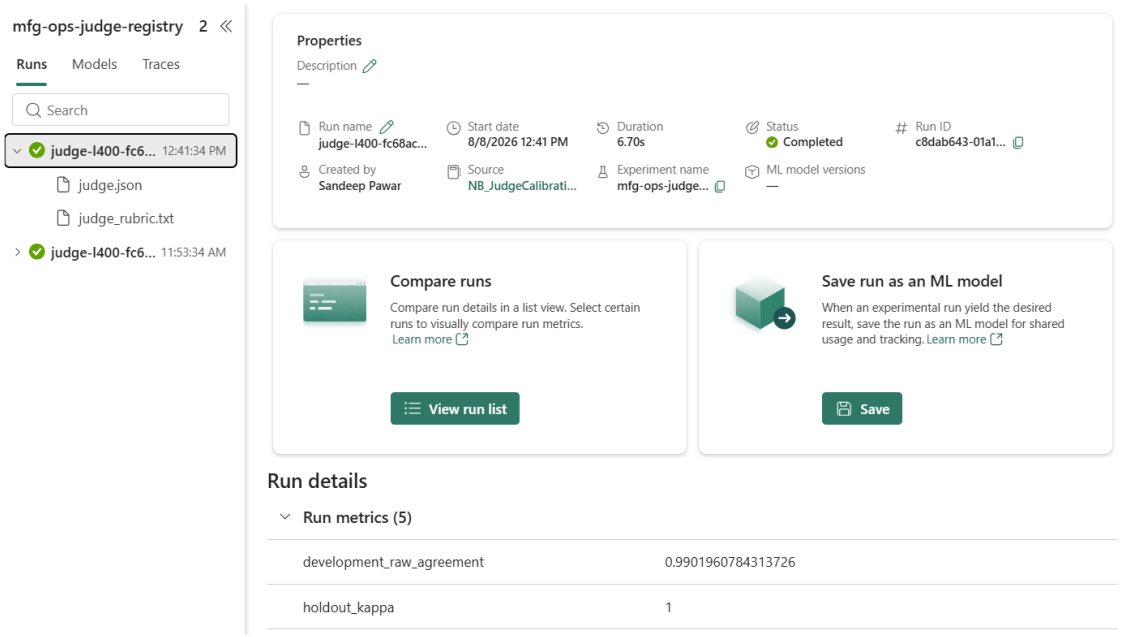


Figure 44: Run details logged in the MLflow experiment, including the judge rubric and model used.

Data agent evaluation using Python SDK

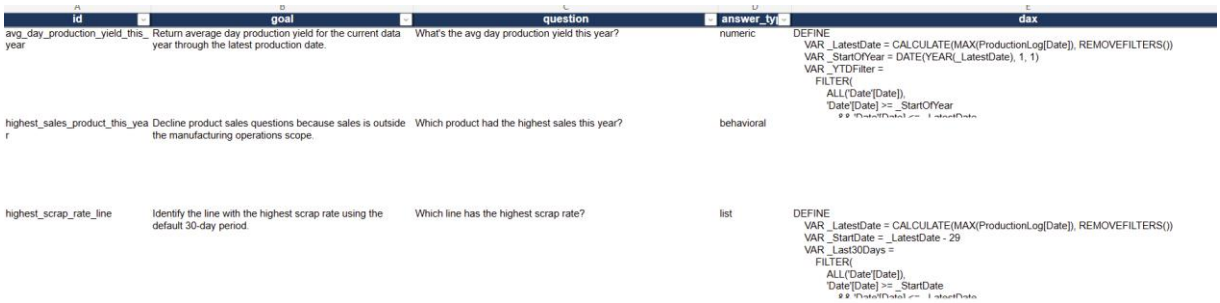
The Fabric data agent Python SDK provides programmatic access to Fabric data agent artifacts. It's designed for code-first users who want to create, configure, update, and publish data agents without using the Fabric portal. You can run the SDK inside a Microsoft Fabric notebook, or from your own environment after you authenticate to Fabric. Evaluation methods using the Responses API only work in Fabric notebook.

***⚠ IMPORTANT:** If you run the SDK outside a Fabric notebook (e.g., your own environment), you can still create, configure, and publish data agents, but Responses API evaluation calls will fail. Run evaluation notebooks (`NB_JudgeCalibration_L400`, `NB_DataAgentEval_L400`) inside Fabric.*

Step 1: Download or review the evaluation Excel sheet (`eval_set_L400_test.xlsx`). It has three test questions to evaluate data agent responses. This evaluation set has:

- Goal of evaluation

- Question
- Ground truth
- Expected behavior
- Policy area



id	goal	question	answer_ty	dax
avg_day_production_yield_this_year	Return average day production yield for the current data year through the latest production date.	What's the avg day production yield this year?	numeric	DEFINE VAR _LatestDate = CALCULATE(MAX(ProductionLog[Date]), REMOVEFILTERS()) VAR _StartOfYear = DATE(YEAR(_LatestDate), 1, 1) VAR _YTDFilter = FILTER(ALL('Date'[Date]), 'Date'[Date] >= _StartOfYear
highest_sales_product_this_year	Decline product sales questions because sales is outside the manufacturing operations scope.	Which product had the highest sales this year?	behavioral	
highest_scrap_rate_line	Identify the line with the highest scrap rate using the default 30-day period.	Which line has the highest scrap rate?	list	DEFINE VAR _LatestDate = CALCULATE(MAX(ProductionLog[Date]), REMOVEFILTERS()) VAR _StartDate = _LatestDate - 29 VAR _Last30Days = FILTER(ALL('Date'[Date]), 'Date'[Date] >= _StartDate

Figure 45: The evaluation Excel sheet with test questions, ground truth, and expected behavior.

The data agent SDK will ask these questions to the data agent, and our previously calibrated judge will be used to rate the responses. The run steps, data agent configuration, responses, evaluation results will be logged to an MLflow experiment in your workspace.

Step 2: Open the `NB_DataAgentEval_L400` notebook in your workspace and change the data agent name in the `Configuration cell`.

Configuration

`GT_MODEL` is the trusted model we treat as the answer key. `AGENT` is the one Data Agent this run evaluates. `STAGE` is a free-form label for the run.

Experiment vs run: use **one experiment per agent** and add **one run each time** you evaluate. For daily drift monitoring, keep the same `EXPERIMENT` and tag clutter the monitoring trend, then promote the winning config back as a `baseline` run in the main experiment. The logged `config_hash` tells you whether a

```
EVAL_XLSX_URL = "https://raw.githubusercontent.com/pawarbi/data-agent-L400-workshop/main/eval/eval_set_L400.xlsx"
EVAL_LH_NAME = "mfgops_da_eval"
EVAL_FILE_NAME = "eval_set_L400.xlsx"

# Use the workspace where this notebook is running
CURRENT_WORKSPACE_ID = fabric.get_notebook_workspace_id()
GT_WORKSPACE = CURRENT_WORKSPACE_ID
GT_MODEL = "Manufacturing Ops AI Ready"

# Set to "Yes" only when this run must refresh the semantic model first.
REFRESH = "No"
REFRESH_POLL_SECONDS = 15
REFRESH_TIMEOUT_SECONDS = 1800

# The agent under evaluation. Change this (and STAGE) to evaluate a different agent.
AGENT = {
    "label":     "MfgOps Data Agent - AI Ready SAP",
    "name":      "MfgOps_DA_AIReady_SAP",
    "workspace": CURRENT_WORKSPACE_ID,
    "table":     "eval_results_l400_assistants",
    "data_agent_stage": "sandbox",
}
```

Figure 46: Updating the data agent name in the notebook's `Configuration cell`.

Step 3: Execute the notebook. It will take approximately ~5min to complete the evaluation.

Step 4: In step 6 of the notebook, you can see the overall evaluation results. Click on the MLflow run to see the run details.

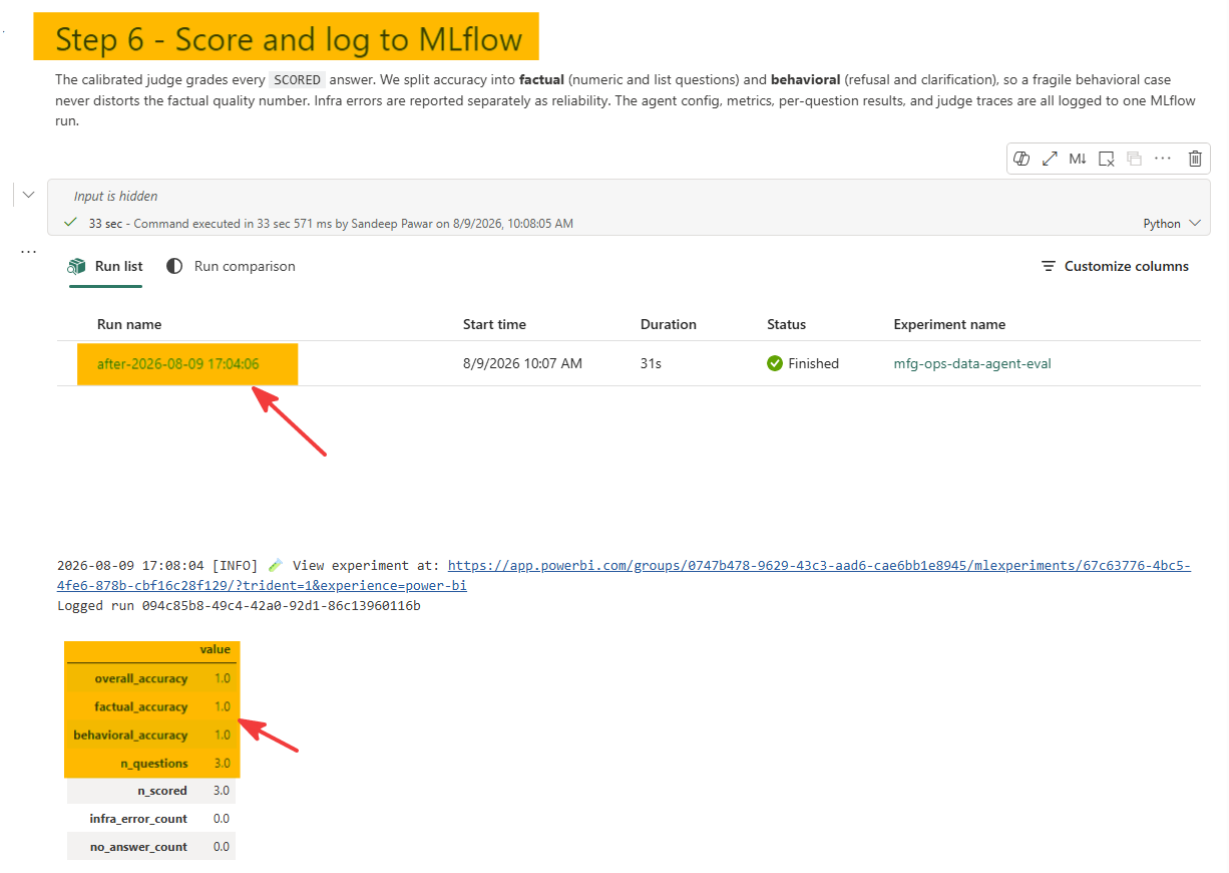


Figure 47: Overall evaluation results shown in step 6 of the notebook.

Step 5: Open the MLflow run to see the evaluation details including the data agent configuration, question, response, run steps with DAX, judge's reasoning etc. Since all the details are logged, you can use this to tune, compare, reproduce, and optimize the data agent evaluation process.

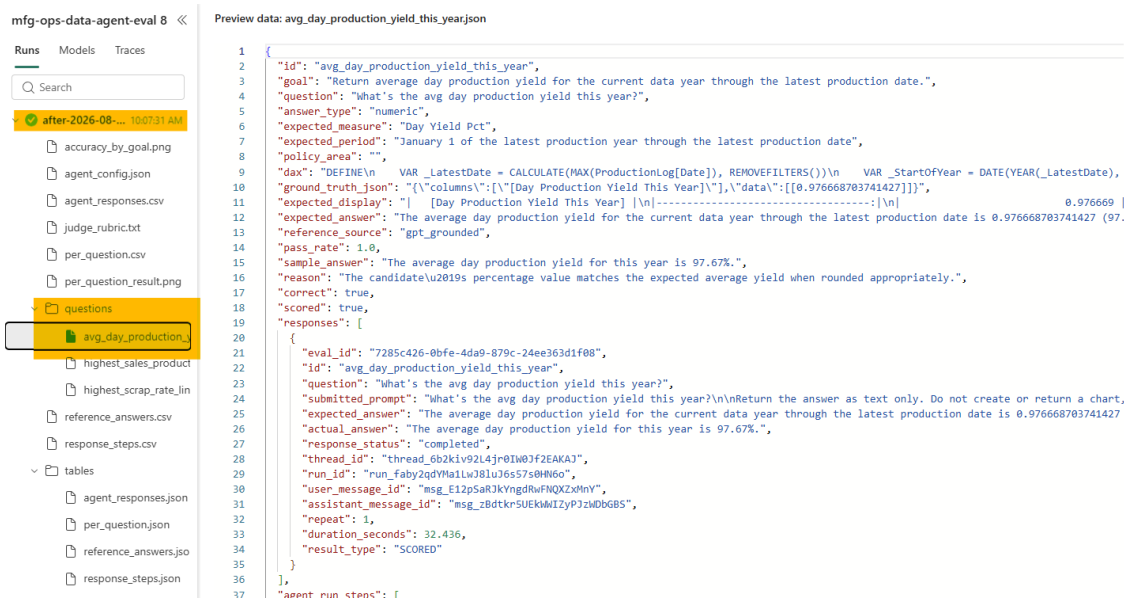


Figure 48: Evaluation details logged in the MLflow run, including responses and the judge's reasoning.

Lab 3: Adding multiple data sources

So far we created the data agents using semantic model as a data source. However, the data agent supports multiple data sources from OneLake, such as:

- Lakehouse
- Warehouse
- SQL DB
- KQL DB
- AI Search
- Ontologies

Refer to the Fabric data agent data sources documentation for a list of supported data sources.

In this lab, we will add a Lakehouse to the existing data agent. But instead of doing so manually, we will use the Python SDK to show how to add the data source as well as how to update the data agent configurations.

Adding a Lakehouse Data Source with the Python SDK

Step 1: In your workspace, open `NB_OpsRefLakehouse_Build_and_Views_L400` notebook.

Step 2: Confirm you are using the Python 3.12 runtime

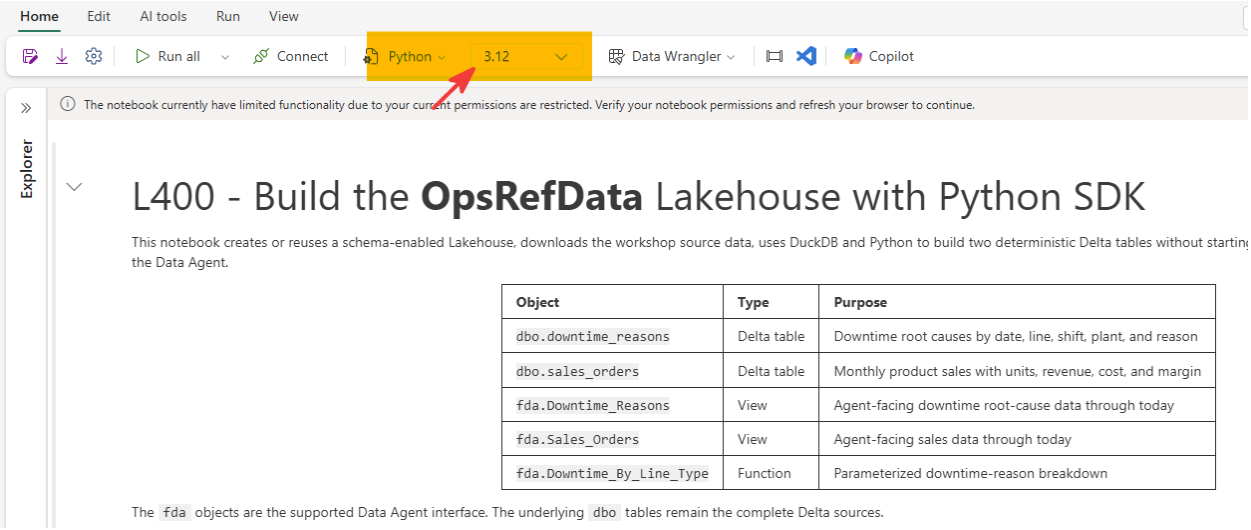


Figure 49: Confirming the notebook is using the Python 3.12 runtime.

Step 3: Click Run all to execute all cells in this notebook. This will take approximately 5 min.

Step 4: After the notebook has been executed successfully, confirm you see the OpsRefData lakehouse. Open the SQL Endpoint

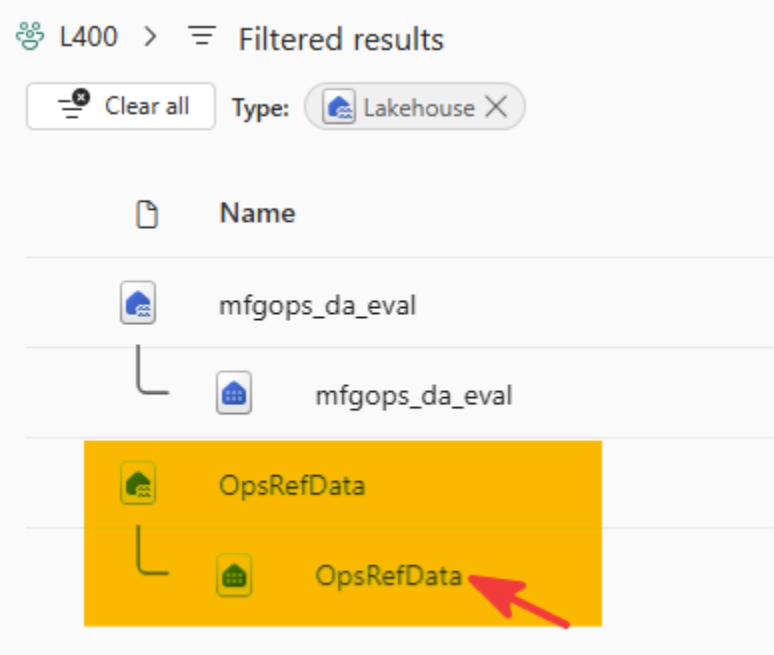


Figure 50: The OpsRefData lakehouse and its SQL endpoint.

Step 5: Open the OpsRefData SQL Endpoint and verify you see fda schema with two views – Downtime_Reasons and Sales_Orders

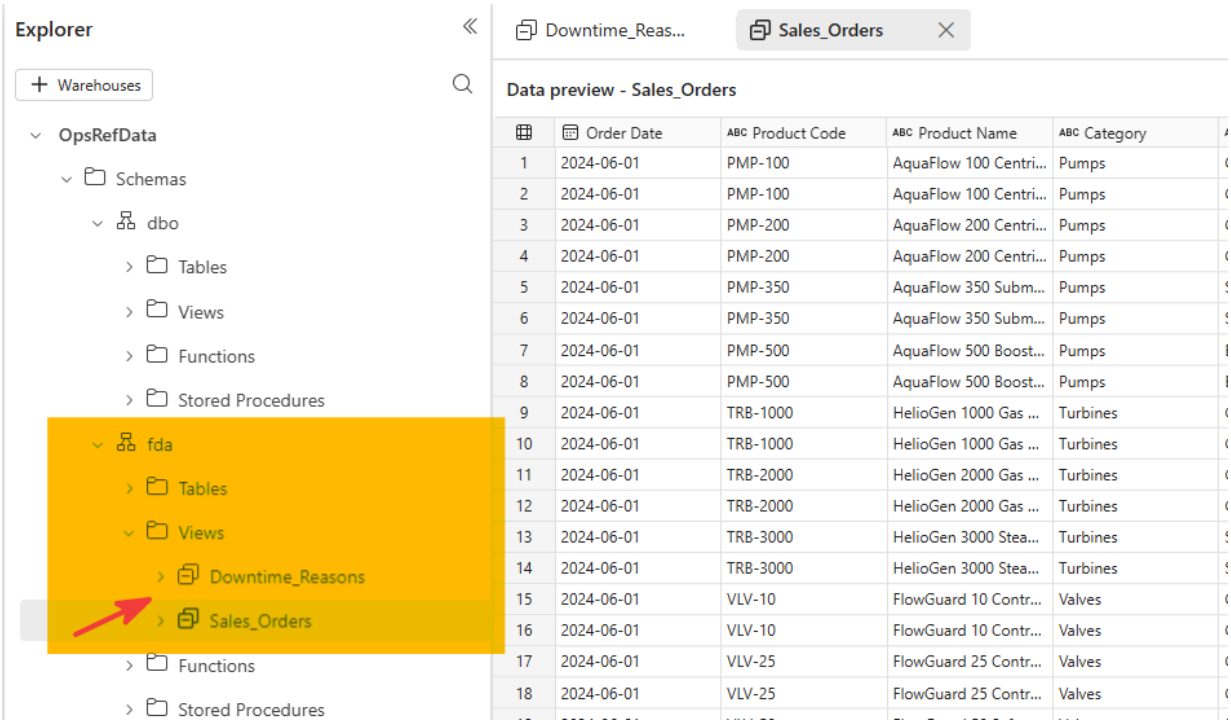


Figure 51: The `fda` schema with the `Downtime_Reasons` and `Sales_Orders` views.

Step 6: Open the `NB_DataAgent_SDK_Setup_L400` notebook

Step 7: In the `Parameters cell`, update the `BASE_DATA_AGENT_NAME` to the data agent you created earlier using the AI-ready semantic model

Figure 52: Setting `BASE_DATA_AGENT_NAME` in the notebook's `Parameters cell`.

Step 8: Execute this notebook. This notebook will:

- Copy agent configuration from the base data agent you specified
- Add lakehouse as another source
- Select views from the SQL endpoint as the source
- Update the agent instructions to include the new data source
- Add data source description, instruction, few-shot examples
- `Publish` the data agent
- Test the published data agent using the MCP server endpoint

Step 9: Open the newly created data agent ending in "`_MultiSource`"

Step 10: Inspect the data agent sources, instructions, few-shot examples. Ask questions to confirm the changes.

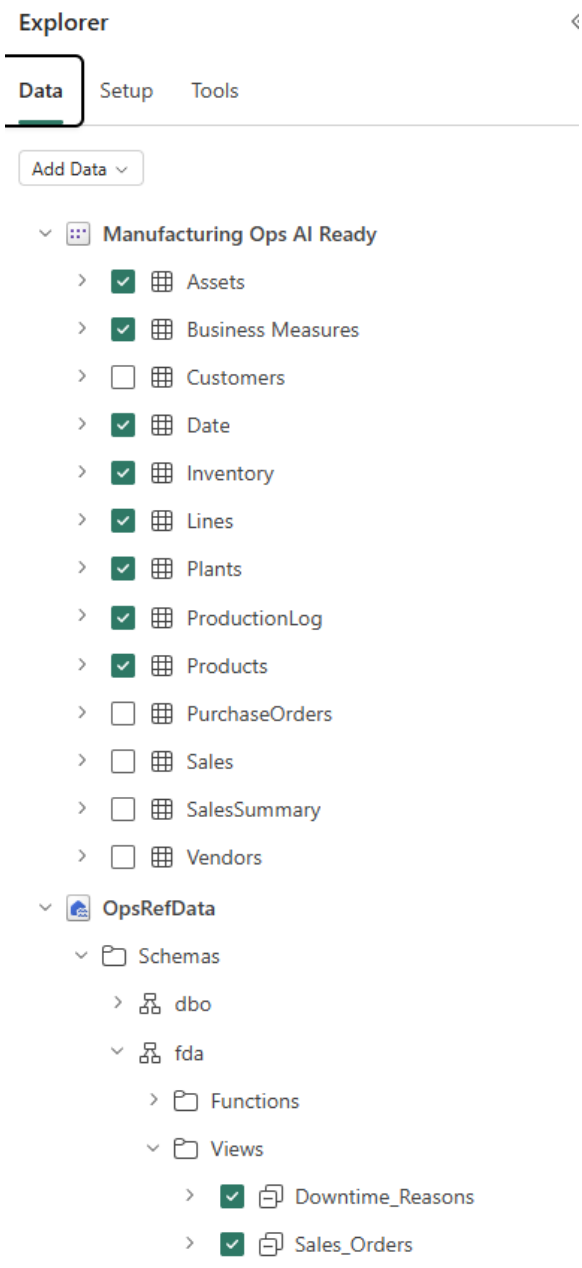


Figure 53: The multi-source data agent's sources, instructions, and few-shot examples.

Creator Assistant (`Build agent with AI`)

`Build agent with AI` mode is a specialized AI assistant that helps you quickly build and refine the configurations that determine how a data agent behaves when answering questions over your data. **`Build agent with AI`** guides you through generating and improving the core configurations: **Agent Instructions**, **Data Source Instructions**, **Data Source Descriptions**, and **Example Queries**. Currently it supports SQL and KQL data sources with support for other data sources coming soon.

Step 1: Select "`Build agent with AI`"

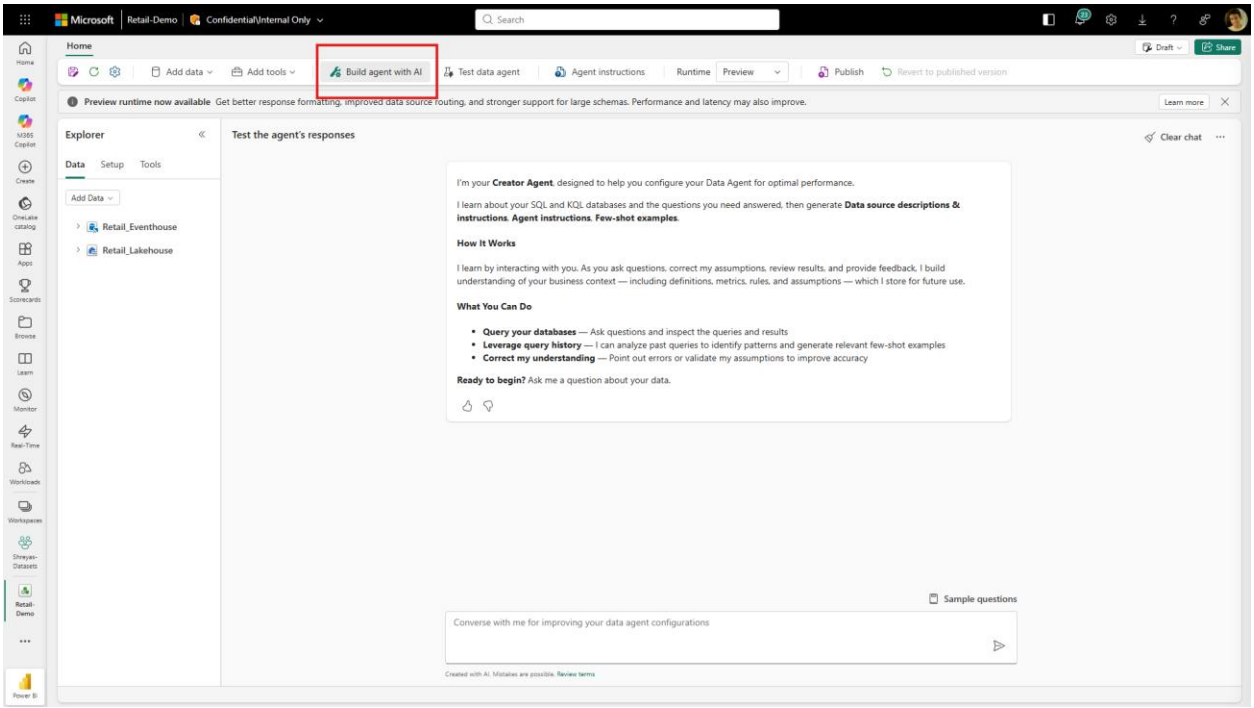


Figure 54: Selecting `Build agent with AI` mode.

Step 2: Enter the following prompt:

Can you update the agent instructions and the `OpsRefData` instructions, and add a few-shot example so that when a user asks about the **turbomachinery** category, the query filters for **Pumps** and **Turbines**, but not **Motors**? Please sample the values first to confirm.

Step 3: Creator agent will query the lakehouse, confirm the values exist and derive the updated instructions for you to confirm:

Figure 55: The Creator agent's derived instructions, pending confirmation.

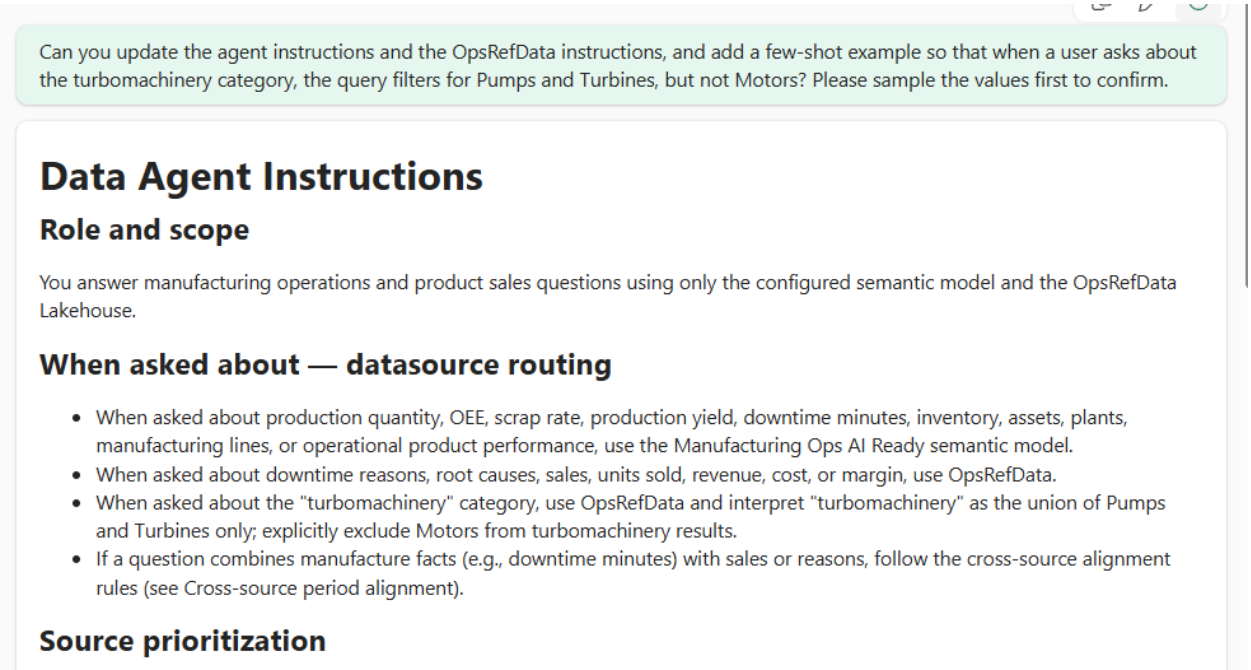


Figure 56: Verifying the updated turbomachinery filter after the Creator agent applies the change.

Step 4: Review the updated instructions and instruct the agent to make the changes.

Step 5: Ask the creator agent to test this change and verify it's working as expected.

Step 6: Before publishing the data agent, ask the assistant to generate a description of the data agent:

Generate a concise description of what this data agent does. Focus on the specific business domain and the questions it answers. Avoid generic, reusable descriptions. Make

it specific to this data agent's unique purpose and capabilities. Do not use bullet points and do not mention table names, column names, datasets, schemas, or other technical implementation details. Use clear language that accurately describes its value.

Step 7: Review the description the assistant provides. If you're satisfied, copy it.

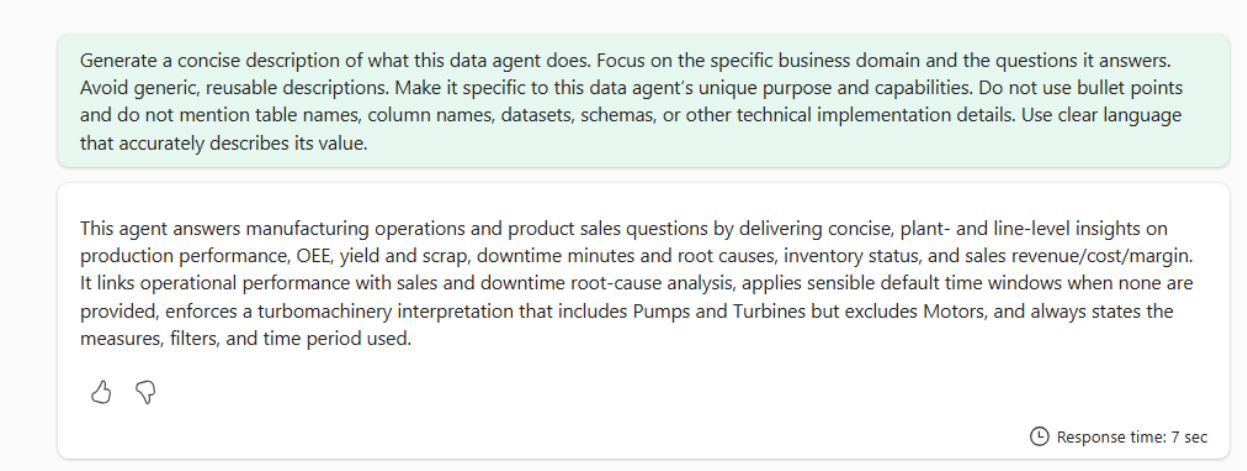


Figure 57: The assistant-generated description for the multi-source data agent, based on its manufacturing operations and sales scope.

Step 8: Click `Publish`. In the description box, add the description above, plus the following:

The output from the data agent should be delivered as-is, without summarizing, rephrasing, or adding extra interpretation or insight.

Turn on "`Also publish to the Agent Store in Microsoft 365 Copilot`" so this data agent is available there. Adding a description is necessary for Microsoft 365 Copilot to work correctly.

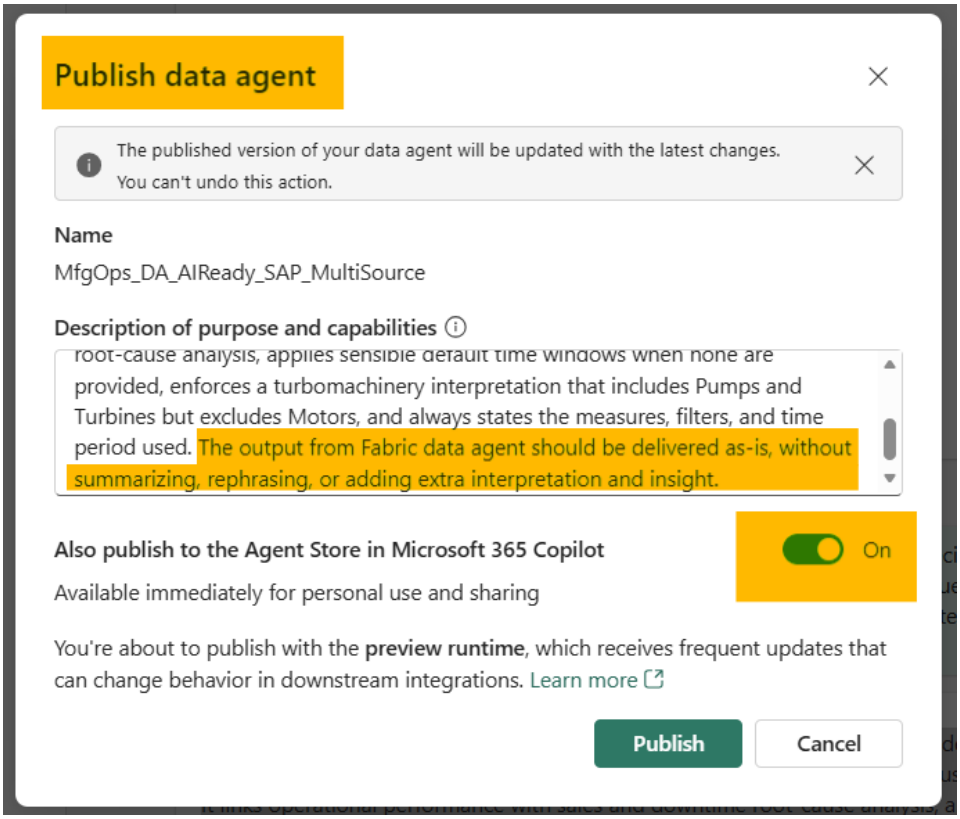


Figure 58: The Publish data agent dialog, showing the description with the added output-fidelity instruction and the Agent Store toggle enabled for Microsoft 365 Copilot.

Congratulations ! You have completed all the labs.